

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 12.09.2026, 18:14:55
Файлов исходного кода: 81
Суммарный объём кода: 1.14 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

ФАЙЛ 1/81: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow\_dispatch:

permissions:

```
deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
+  pages: write
+  id-token: write
+
+concurrency:
```

Универсальное пособие • полный исходный код

```
-----
+       name: github-pages
+       url: ${ steps.deployment.outputs.page_url }
+     steps:
+       - name: Deploy
+         id: deployment
+         uses: actions/deploy-pages@v5
+     ....

## `github/workflows/rebuild-pdf.yml`

### Полное содержимое после изменений

```yaml
name: Rebuild code PDF

Пайплайн пересборки PDF при каждом коммите:
#
исходный код → full_code_all_pages.txt (script)
|
|→ page-0001.png ... page-NNNN.png (
|
|→ full_code_all_page
#
Готовые TXT и PDF коммитаются обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.

on:
 push:
 branches:
 - "**"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бескон
 - "public/export/**"
 - "**/*.md"
 workflow_dispatch:
 inputs:
 density:
 description: "DPI растеризации страниц (по умол
```

- ```
        sudo sed -i 's/rights="none" pattern="\{P
    fi
done
convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
EXPORT_DENSITY: \${github.event.inputs.density
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
{
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \public/export/full_code_all_pages
 echo "| \public/export/full_code_all_pages
 echo "| PNG-страниц | \${ls tmp/export-pages.
} >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7
with:

```
- name: Checkout
- uses: actions/checkout@v4
+ uses: actions/checkout@v6
  with:
    fetch-depth: 0
    persist-credentials: true

- name: Setup Node.js
- uses: actions/setup-node@v4
+ uses: actions/setup-node@v7
  with:
-   node-version: "22"
+   node-version: "24"
    cache: npm

- name: Install ImageMagick + DeJaVu fonts
@@ -97,7 +97,7 @@ jobs:
    } >> "$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)
- uses: actions/upload-artifact@v4
+ uses: actions/upload-artifact@v7
  with:
    name: full-code-export
    path: |
@@ -107,7 +107,7 @@ jobs:
    retention-days: 30

- name: Upload artifacts (PNG-страницы)
- uses: actions/upload-artifact@v4
+ uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
....

## `index.html`
```

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

Patch относительно `main`

````diff

diff --git a/public/favicon.svg b/public/favicon.svg

new file mode 100644

index 00000000..3bbbba0

--- /dev/null

+++ b/public/favicon.svg

@@ -0,0 +1,5 @@

+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"

+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>

+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>

+ <circle cx="47" cy="21" r="4" fill="#34d399"/>

+</svg>

````

`src/App.tsx`

Полное содержимое после изменений

````tsx

import { useCallback, useEffect, useMemo, useState } from "react";

import { chapters } from "../data/content";

import { Navbar } from "../components/Navbar";

import { Sidebar } from "../components/Sidebar";

import { MobileToc } from "../components/MobileToc";

import { ChapterView } from "../components/ChapterView";

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/SimulatorModal";



```
<Navbar activeTab={activeTab} setActiveTab={setActiveTab}
```

```
{activeTab === "guide" ? (
```

```
<>
```

```
<MobileToc
```

```
 selectedChapterId={activeChapterId}
```

```
 setSelectedChapterId={goTo}
```

```
 currentTitle={activeChapter.title}
```

```
 index={index}
```

```
 total={chapters.length}
```

```
/>
```

```
<div className="flex flex-col lg:flex-row max
```

```
 /* Сайдбар только на десктопе — на мобильном
```

```
<div className="hidden lg:block lg:w-80 shr
```

```
 <Sidebar selectedChapterId={activeChapterId}
```

```
</div>
```

```
<main id="main-content" className="flex-1 m
```

```
 /* Шапка главы с быстрыми переходами */
```

```
<div className="flex items-center justify
```

```
 <div className="min-w-0">
```

```
 <div className="text-[10px] uppercase
```

```
 {activeChapter.category || "Универс
```

```
</div>
```

```
<h2 className="text-base sm:text-xl f
```

```
 {activeChapter.title}
```

```
</h2>
```

```
</div>
```

```
<ChapterNav prev={prev} next={next} ind
```

```
</div>
```

```
<div className="h-1 w-full bg-slate-800 r
```

```
 <div
```

```
 className="h-full bg-gradient-to-r fr
```

```
 style={{ width: `${progress}%` }}
```

```
/>
```

```
</div>
```

-----  
 ### Patch относительно `main`

```diff

diff --git a/src/App.tsx b/src/App.tsx

index 2091f33..4583756 100644

--- a/src/App.tsx

+++ b/src/App.tsx

@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/SimulatorModal";

import { SimulatorHub } from "../components/SimulatorHub";

+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {

- const [activeTab, setActiveTab] = useState<"guide" | "simulator">("guide");

+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">("guide");

const [activeChapterId, setActiveChapterId] = useState<string>("");

const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {

</main>

</div>

</>

-) : (

+) : activeTab === "simulator" ? (

<main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">

<SimulatorHub

onOpen={setModalVizId}

@@ -105,6 +106,8 @@ export default function App() {

}}

/>

</main>

+) : (

+ <PythonCompiler />

}}

<SimulatorModal vizId={modalVizId} onClose={() => setModalVizId("")} />

```

```
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-extrabold">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto" >
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
 </button>
 <button
 id="tab-compiler-btn"
 onClick={() => setActiveTab('compiler')}
 >
```

```

er:bg-amber-600/20 border border-amber-500/
title="Скачать всё пособие одним автономным
>
 {busy ? <Loader2 className="w-4 h-4 animate
</button>
</div>
</div>
</header>
);
};
....
```

### Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components.
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

  export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
    >
      <Cpu className="w-4 h-4" /> Тренажёры
    </button>
```

```
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) =>
  setStderr: (options: { batched: (message: string) =>
  setStdin: (options: { stdin: () => string | number |
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promi
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_I
      (module as PyodideModule).loadPyodide({ indexURL:
    });
  }
  return runtimePromise;
}

function errorText(error: unknown) {
  if (error instanceof Error) return error.message;
  return String(error);
}
```

```

    }, [code, stdin, running, runtimeReady]);

const reset = () => {
  setCode(STARTER_CODE);
  setStdin("");
  setOutput("Нажмите «Запустить», чтобы увидеть резул
});

return (
  <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
    <div className="max-w-6xl mx-auto">
      <div className="flex flex-col sm:flex-row sm:items-center">
        <div>
          <div className="inline-flex items-center gap-2">
            <Terminal className="w-4 h-4" /> Боковая панель
          </div>
          <h2 className="text-2xl sm:text-3xl font-extrabold">
            <p className="text-slate-400 mt-2 max-w-2xl">
              Пишите небольшой код и запускайте его прямо в браузере,
              который переводит CPython в WebAssembly.
            </p>
          </div>
          <a
            href="https://pyodide.org/"
            target="_blank"
            rel="noreferrer"
            className="inline-flex items-center gap-1.5"
          >
            Как это работает <ExternalLink className="w-4 h-4" />
          </a>
        </div>

        <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]">
          <section className="rounded-2xl border border-slate-200 p-4">
            <div className="flex flex-wrap items-center gap-2">
              <div className="flex items-center gap-2">
                <span className="w-2.5 h-2.5 rounded-full bg-slate-200">
                <span className="text-sm font-bold text-slate-700">

```

```

        />

        <div className="border-t border-slate-800 p-4">
          <label className="block px-4 pt-3 text-slate-700">
            Ввод для input() • по одной строке на к
          </label>
          <textarea
            id="python-stdin"
            value={stdin}
            onChange={(event) => setStdin(event.target.value)}
            spellCheck={false}
            rows={2}
            placeholder={'Например:\nАлиса'}
            className="block w-full resize-y bg-transparent border border-slate-800"
          />
        </div>
      </section>

      <section className="rounded-2xl border border-slate-800 p-4">
        <div className="flex items-center justify-between">
          <div className="flex items-center gap-2">
            <Terminal className="w-4 h-4 text-emerald-500" />
          </div>
          <span className={`inline-flex items-center gap-2`} >
            {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
            {runtimeReady ? "готов" : "не загружен"}
          </span>
        </div>
        <pre className="min-h-[360px] max-h-[560px] p-4 font-mono text-slate-300">
          {output}
        </pre>
        <div className="border-t border-slate-800 p-4">
      </section>
    </div>

    <div className="mt-5 grid md:grid-cols-3 gap-3">

```

```

-----
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>
+  setStdout: (options: { batched: (message: string) =>
+  setStderr: (options: { batched: (message: string) =>
+  setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Prom
+});
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE
+      (module as PyodideModule).loadPyodide({ indexURL
+    });
+  }
+  return runtimePromise;
+}
+
+function errorText(error: unknown) {
+  if (error instanceof Error) return error.message;
+  return String(error);
+}
+
+export function PythonCompiler() {
+  const [code, setCode] = useState(STARTER_CODE);
+  const [stdin, setStdin] = useState("");
+  const [output, setOutput] = useState("Нажмите «Запус
+  const [running, setRunning] = useState(false);
+  const [runtimeReady, setRuntimeReady] = useState(fal
+  const [runtimeMessage, setRuntimeMessage] = useState

```



```

+
+ return (
+   <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+     <div className="max-w-6xl mx-auto">
+       <div className="flex flex-col sm:flex-row sm:items-center">
+         <div>
+           <div className="inline-flex items-center gap-2">
+             <Terminal className="w-4 h-4" /> Боковая панель
+           </div>
+           <h2 className="text-2xl sm:text-3xl font-extrabold">
+             <p className="text-slate-400 mt-2 max-w-2xl">
+               Пишите небольшой код и запускайте его прямо в браузере,
+               который переводит CPython в WebAssembly.
+             </p>
+           </div>
+           <a
+             href="https://pyodide.org/"
+             target="_blank"
+             rel="noreferrer"
+             className="inline-flex items-center gap-1.5"
+           >
+             Как это работает <ExternalLink className="text-slate-400" />
+           </a>
+         </div>
+
+       <div className="grid xl:grid-cols-[minmax(0,1fr),minmax(0,1fr)]">
+         <section className="rounded-2xl border border-slate-200 p-4">
+           <div className="flex flex-wrap items-center gap-2">
+             <div className="flex items-center gap-2">
+               <span className="w-2.5 h-2.5 rounded-full bg-slate-200">
+                 <span className="text-sm font-bold text-slate-400">
+                   <span className="text-[11px] text-slate-400">
+                     </div>
+                 <div className="flex items-center gap-2">
+                   <button
+                     type="button"
+                     onClick={reset}
+                     className="inline-flex items-center gap-1.5"

```

```

+         id="python-stdin"
+         value={stdin}
+         onChange={(event) => setStdin(event.target.value)}
+         spellCheck={false}
+         rows={2}
+         placeholder={'Например:\nАлиса'}
+         className="block w-full resize-y bg-tran
ceholder:text-slate-700"
+       />
+     </div>
+   </section>
+
+   <section className="rounded-2xl border border-slate-200" >
+     <div className="flex items-center justify-between" >
+       <div className="flex items-center gap-2" >
+         <Terminal className="w-4 h-4 text-emerald-500" />
+       </div>
+       <span className={`inline-flex items-center gap-2`} >
+         <CheckCircle2 className="w-4 h-4 text-emerald-500" />
+         {runtimeReady ? "готов" : "не загружен"}
+       </span>
+     </div>
+     <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+       {output}
+     </pre>
+     <div className="border-t border-slate-800 p-4" >
+   </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3" >
+   <div className="flex gap-2 rounded-xl border border-slate-200" >
+     <Info className="w-4 h-4 shrink-0 text-indigo-500" />
+     <span>Ctrl или Cmd + Enter запускает код. Ищет файл python.py в папке
+   </div>
+   <div className="flex gap-2 rounded-xl border border-slate-200" >
+     <Info className="w-4 h-4 shrink-0 text-amber-500" />
+     <span>Первый запуск скачивает примерно нес

```

```

server: {
  host: "0.0.0.0",
  port: 5173,
  strictPort: true,
  // предпросмотр может открываться через внешний про
  allowedHosts: true,
},
preview: {
  host: "0.0.0.0",
  port: 4173,
  strictPort: true,
  allowedHosts: true,
},
});

```

Patch относительно `main`

```

````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {

```

Универсальное пособие • полный исходный код

-----  
- "Душные" детали (код, формулы, сложные доказательства)  
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы и классы в файлах `src/data/content.ts`. Не используй чистый markdown.

-----  
ФАЙЛ 4/81: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512 Б

-----  
<!doctype html>  
<html lang="ru" class="dark bg-slate-950 text-slate-100">  
 <head>  
 <meta charset="UTF-8" />  
 <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />  
 <title> Дискретка для тупых – Интерактивный гид по дискретке  
 </head>  
 <body class="bg-slate-950 text-slate-100 min-h-screen">  
 <div id="root"></div>  
 <script type="module" src="/src/main.tsx"></script>  
 </body>  
</html>

-----  
ФАЙЛ 5/81: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312 Б

-----  
{  
 "name": "Remix Remix: ExamPrep Reader",  
 "description": "Удобная web-читалка для подготовки к экзаменам",  
 "requestFramePermissions": [],  
 "majorCapabilities": [  
 "MAJOR\_CAPABILITY\_SERVER\_SIDE\_GEMINI\_API"  
 ]  
}

## Универсальное пособие • полный исходный код

```

 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
},
 "description": "Интерактивное пособие по алгоритмам и
}
```

-----  
ФАЙЛ 7/81: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР:

-----  
# algv0 – Универсальное пособие по алгоритмам и структурам

Интерактивная web-читалка (React + Vite + Tailwind) с синтаксисом  
и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash  
npm install  
npm run dev          # предпросмотр на http://0.0.0.0:5173  
npm run build        # сборка в dist/ (single-file)  
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в один файл

| Команда              | Шаг       | Результат                                               |
|----------------------|-----------|---------------------------------------------------------|
| ---                  | ---       | ---                                                     |
| `npm run export:txt` | код → txt | `public/export/full-code.txt`                           |
| `npm run export:png` | txt → png | `tmp/export-pages/00000000000000000000000000000000.png` |
| `npm run export:pdf` | png → pdf | `public/export/full-code.pdf`                           |
| `npm run export`     | всё сразу | txt + png + pdf                                         |

## Универсальное пособие • полный исходный код

-----  
подчёркиваются, а по наведению/тапу показывается карта  
мини-анимацией и кнопкой «Показать симулятор» (симуля  
Словарь — `src/data/glossary.ts`, мини-анимации — `src  
\* **Реестр интерактивов** — `src/components/vizRegistry`  
и для панели под главой, и для модалки из подсказки.

### ## Структура

---

src/

|                    |                                   |
|--------------------|-----------------------------------|
| App.tsx            | — оболочка, привязка глав к визу. |
| components/        | — интерактивные симуляторы (Дейк  |
| data/              | — контент пособия (главы/билеты   |
| scripts/export/    | — пайплайн код → txt → png → pdf  |
| public/export/     | — сгенерированные TXT и PDF (обн  |
| .github/workflows/ | — автоматизация                   |

---

---

### # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструмен  
азуемые теги и классы.

### ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и
- \* Подзаголовки: Используют тег `

### `.

### ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика  
параграфы `

`.

### ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формул  
k).

Универсальное пособие • полный исходный код

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава              | id        | Что делать           |
|--------------------|-----------|----------------------|
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| Билет  | Тема                           | Статус                      |
|--------|--------------------------------|-----------------------------|
| **1**  | Дерево отрезков (Segment Tree) | Главы нет. К                |
| **7**  | Планарность и раскраска графов | Номерной гла                |
| **11** | Мосты в графах                 | Номерной главы нет; есть бе |
| **13** | Эйлеровы пути и циклы          | Номерной главы нет; с       |

Дополнительно – «осиротевшие» визуализаторы без глав (к

- \* `stack-dfs` → `StackViz.tsx` – \*\*Стек и DFS\*\*
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – \*\*Очередь
- \* `heap-beam-search` → `HeapViz.tsx` – \*\*Куча и лучевой
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` – \*\*Дер
- \* `dynamic-programming` → `DPVisualizer.tsx` – \*\*Динами

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

| Глава                                  | id           | Комментарий     |
|----------------------------------------|--------------|-----------------|
| Планарность: K5, K3,3 и формула Эйлера | `planarity-e | т «на пальцах». |

## Универсальное пособие • полный исходный код

|    |                                      |   |  |       |
|----|--------------------------------------|---|--|-------|
| 18 | 18. Графы. Остовное дерево. Краскал  | 1 |  | -     |
| 19 | 19. Графы. Остовное дерево. Прима    | 1 |  | -     |
| 20 | 20. Графы. Остовное дерево. Борувка  | 1 |  | -     |
| 21 | 21. Префикс-функция (π), КМП         | 4 |  | -   - |
| 22 | 22. Z-функция                        | 4 |  | -   - |
| 23 | 23. Алгоритм Ахо–Корасик             | 2 |  | -   - |
| 24 | 24. Классы сложности, сведение задач | 4 |  | -   - |
| -  | Обзор: Кратчайшие пути и Графы       | - |  | -   - |
| -  | Алгоритмы на карте                   | - |  | -   - |
| -  | Эйлер: Путь ≠ Цикл                   | - |  | -   - |
| -  | Мосты в графах: код + визуализация   | 1 |  | -   - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить (`segment-trees`, `stack-dfs`, `queue-bfs`, `heap-be...`)  
это 5 готовых компонентов, которые сейчас просто не...
2. **\*\*Дописать аналогии\*\*** в 5 главам из «Уровня 3» (по ш...
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» – минимум inli...
4. **\*\*Решить судьбу `alg-map`\*\*** – раскрыть или удалить.

ФАЙЛ 9/81: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6...

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],
```



```
// https://vite.dev/config/
export default defineConfig({
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

---

ФАЙЛ 11/81: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 КБ

---

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
```

```

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе — на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 m-0 p-0" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0" >
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded" >
 <div
 className="h-full bg-gradient-to-r from-slate-800 to-slate-900"
 >

```

---

ФАЙЛ 12/81: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 KB

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)\[\]]/g, "\\$1");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
 const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l));
 // границы «не буква/цифра» – \b не работает с кириллицей
 return new RegExp(`(?<![\\p{L}\\p{N} _-])(${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * , чтобы на
 * повесить всплывающую карточку (как превью ссылок в Википедии)
 */
```

```

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) {
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);
 }

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
 const run = () => {
 const el = ref.current;
 if (!el) return;
 try {
 annotateTerms(el);
 } catch {
 //
 }
 };
 useEffect(run, []);
}

```

```
 display: none;
 }
 .no-scrollbar {
 scrollbar-width: none;
 }
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
 from {
 transform: translateX(-100%);
 }
 to {
 transform: translateX(0);
 }
}

@keyframes hintIn {
 from {
 opacity: 0;
 transform: translateY(4px);
 }
}
```

```
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}

.chapter-body :where(pre, code) {
 white-space: pre-wrap;
 word-break: break-word;
}

.chapter-body pre {
 overflow-x: auto;
 max-width: 100%;
 font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
 display: block;
 width: 100%;
 overflow-x: auto;
 border-collapse: collapse;
 font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
 cursor: pointer;
 padding: 0.35rem 0;
}

@media (max-width: 767px) {
 .chapter-body :where(.grid) {
 grid-template-columns: minmax(0, 1fr) !important;
 }
 .chapter-body :where(h2) {
 font-size: 1.25rem !important;
 line-height: 1.3 !important;
 }
}
```

```
 width: 0%;
 }
 to {
 width: 100%;
 }
}
```

---

ФАЙЛ 14/81: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/81: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

```

export const additionalTickets: Chapter[] = [
 {
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные за
 ь отрезок длины 7 (перекрывая друг друга по
 ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb-4">

```



двоичная куча. Пара — точка на декартовой

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Постр
платит $\approx n \cdot \log n$ сравнений, counting по ма
точка вставляется спуском от корня: $x \leq$ уз
динственно. Весь процесс — в симуляторе под
«соединить сам» — кликаешь две точки,
ь родитель», «получился бы цикл», «равные »,
«Миф $2k/2k+1$ » и «Поиск $\neq$ сортировка»
ждый запрещённый ход.</p>
```

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">
 Й
ext-2xl text-slate-300">Й
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Split — таможня
 </div>
 <p class="text-slate-300 text-sm">Sp
х»». Спуск от корня: узел, который вместе
скрытия — досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Merge — стыковка
 </div>
 <p class="text-slate-300 text-sm">M
```

```

 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}
}

</div>
</details>

<details class="bg-slate-900/40 rounded-lg" >
 <summary class="p-4 font-bold text-slate-300" >
 -b border-slate-700/50">
 Код: вставка, merge, erase (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300" >
 <pre class="bg-slate-950 p-4 rounded" >
def insert(root, key, pri):
 if root is None:
 return Node(key, pri)
 if key <= root.key: # равные x — в левую ветвь
 root.left = insert(root.left, key, pri)
 else:
 root.right = insert(root.right, key, pri)
 return root

for key, pri in pairs_sorted_by_pri_desc: # порядок э
 root = insert(root, key, pri)

 <pre class="bg-slate-950 p-4 rounded" >
def merge(a, b): # все x в a < все x в b
 if not a or not b: return a or b
 if a.pri > b.pri:
 a.right = merge(a.right, b); return a
 b.left = merge(a, b.left); return b

def erase(root, key): # удалить произвольный элемент
 if key < root.key: root.left = erase(root.left, key)
 elif key > root.key: root.right = erase(root.right, key)
 else: root = root.left or root.right
 return root

```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
```

```
<p class="text-slate-300 text-sm">Ч.
дерево временно разрешено растить кривым, з
да живёт в амортизированном $O(\log n)$.
```

```
</div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Почем
от «двух зигов подряд» — а формы-то разные.
от разложен на кадры — прицел на ребро →
зелёной подсветкой переехавших рёбер. Рядом
ению вывиха: одна кость ходит вокруг другой
ту за клик, и проверка скажет, совпал ли тв
```

```
<div class="bg-slate-900/40 rounded-lg border">
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
ое; два нижних зига оставили бы почти ту же
 <p class="text-slate-400 text-sm mb-1">
а к верху, симулятор подсвечивает переехавши
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4">с
ое <code class="bg-slate-950 px-2 py-0.5 rounded">
одиночная операция — до <code class="bg-sla
> $O(n)$ </code>. Все операции через один splay
ое) → вырезать корень → merge двух половин
```

```
<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
```

```

 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 × Аналогия 1: Вырезание прямоугольника
 </div>
 <p class="text-slate-300 text-sm mb-4">
 з него маленький целевой прямоугольник в центре.
 ый верхний угол отрезается дважды! Поэтому формула
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `

```

```

<h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
 еская сортировка – это расписание, которое
 >
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. А

```

```

<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и $f_{up}[v] > tin[u]$.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
order border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
этот мост, экономикой обоих кусков придет
 </div>
 </div>
 </div>
 </div>
</div>
</section>`,
},
{
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает связность",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

```

```

 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрыв
 </div>
 <p class="text-slate-300 text-sm mb
 е сходится нечетное число линий, значит, из
 а везде должно быть четное число (зашел-выш
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "pathfinding-accel",
 title: "17. Графы. Ускорения поиска",
 type: "html",
 description: "",
 category: "Графы. Пути",
 content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-r
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="bg-slate-900 p-4 rounded-lg mb-r
 <p class="text-lg text-blue-300 italic m
 <p class="text-xl font-mono text-white">
 d-длина.</p>
 </div>
 </div>
 </div>
`
 }
]

```

```

 order border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-6">
с самых дешевых дорог в стране". Он строит
единую сеть.</p>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "mst-prima",
 title: "19. Графы. Основное дерево. Прима",
 type: "html",
 description: "",
 category: "Графы. Основные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 order border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>

```



```

 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 i]), который одновременно является её суффиксом.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс и суффикс)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 идишь "АБРА". Если ты ошибешься при поиске
 концовка "АБРА" совпадает с началом "АБРА",
 то тебе придется провести много перепроверок!</p>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
]

```

```

 <p class="text-xl font-mono text-white">
щегося с i.</p>
</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Копипаста
 </div>
 <p class="text-slate-300 text-sm mb
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной коп
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",

```

```
 </div>
</section>`,
 },
];
```

---

ФАЙЛ 18/81: src/data/content\_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 385 | РАЗМ

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html";
 content: string;
}

export const chapters: Chapter[] = [
 {
 id: "euler-path-vs-cycle",
 title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
 type: "html",
 content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
 <span class="bg-indigo-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">He by
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
```

```

ng> ровно раз и вернуться домой.
 Ты обходишь все бары (вершины) ровно
 Главное – зайти и выйти, не заходя
 Никто не знает, как решать быстро.

 лноту).
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-400
 order-slate-700/50">
 Душный режим: Формальное доказательство
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
 <p class="text-blue-400">Теорема Эйлера
 <p>
 Связный граф содержит эйлеров цикл ⇔

Эйлеров путь (не цикл) ⇔ ровно 2 вершины
 </p>
 <p class="text-rose-400">Почему Гамильтон?
 <p>
 Задача коммивояжёра (TSP) – это взрыв мозга.
 Доказано, что любой полиномиальный алгоритм
 В общем, не парься. Просто запомни!
 </p>
 </div>
</details>
</div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "Мосты в графах: код + визуализация",
 type: "html",
 content: `<section id="bridges-code" class="mb-20 scroll-mt-20">
 <div class="flex items-center mb-6">

```

```

 </p>
 </div>

```

```

<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-rose-500 shadow-md">

```

Аналогия: Кровеносный сосуд

```

 </div>

```

```

 <p class="text-slate-300 text-sm mb-4">

```

Граф – это кровеносная система. Ребра (артерии и вены) – это не мост, живём.

Если же перерезать единственную артерию, то наступит катастрофа.

Алгоритм DFS ищет такие «критические» ребра.

```

 </p>

```

```

 </div>

```

```

</div>

```

```

<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
 <summary class="p-4 font-bold text-slate-400">
 Полный код на C++ (Скрыто)
 </summary>

```

Полный код на C++ (Скрыто)

```

 </summary>

```

```

 <div class="p-5 text-sm text-slate-300 space-y-2">
 <p>Классическая реализация – <code class="text-slate-400">
 оходов.</p>

```

```

 <div class="bg-slate-950 p-4 rounded-lg border-rose-500 shadow-md">

```

```

 <pre class="text-xs font-mono text-slate-400">

```

```

#include <vector>;

```

```

#include <algorithm>;

```

```

using namespace std;

```

```

int n; // число вершин

```

```

vector<vector<int>>> adj; // список смежности

```

```

vector<bool> visited;

```

```

vector<int> tin, low;

```

```

int timer;

```

```

 </div>
</section>`,
 },
 {
 id: "planarity-euler-formula",
 title: "Планарность: K5, K3,3 и формула Эйлера",
 type: "html",
 content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Плана
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic m
 <p class="text-3xl font-mono text-white"
 <div class="text-sm text-slate-400 spac
 <p><span class="text-blue-400 font-l
 <p><span class="text-emerald-400 fo
 <p><span class="text-rose-400 font-l
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше — граф гарантированно
 </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
 order-green-500 shadow-md">

```

```


Каждая грань ограничена минимум

Сумма длин граней = $2E = 20$.

Отсюда: $2E \geq 3F \Rightarrow 20 \geq 21$ — <sp

Значит, K_5 не может быть планарн
 </p>
 </div>
</details>
</div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 content: `<section id="graph-articulation" class="ml
<div class="flex items-center mb-6">
 <span class="bg-rose-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Точки
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400 r

 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-rose-300 italic r
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">

```

<p>

Для стартовой вершины обхода правил  
то выше неё прыгать некуда). Поэтому для не  
у него <strong>больше 1 независимого ребенка

</p>

<div class="bg-slate-950 p-4 rounded-lg

<pre class="text-xs font-mono text-

```
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
 if (to == p) continue;
 if (visited[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
 }
 if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}
```

</div>

</div>

</details>

<div class="bg-indigo-950/60 p-6 rounded-xl bor

<h4 class="text-lg font-bold text-indigo-400

<p class="text-slate-300 text-sm mb-4">

Это глубокий дуальный мост между ориент

</p>

<ul class="list-disc list-inside text-sm te

<li><strong class="text-rose-400">Точки



```

// Remove old 7, 11, 12, 13: graph-planar-colors, graph
const toRemove = ["graph-planar-colors", "graph-bridges

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
 if (!toRemove.includes(c.id)) {
 // We prefer the newer versions (like tickets14_20
 dedup.set(c.id, c);
 }
});

// Add the new ones
if (newChapters) {
 newChapters.forEach(c => dedup.set(c.id, c));
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
 const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
 return numA - numB;
});

export const chapters: Chapter[] = finalChapters;

export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
 "intro", "dijkstra", "bellman-ford", "floyd", "aho-corasi
 "mst-kruskal", "mst-prima", "mst-boruvka", "string-kmp",
 "string-z-func", "segment-trees", "sparse-table", "dynam
];

```

```

 demo: "bfs",
 simulator: "queue-bfs",
},
{
 id: "dfs",
 labels: ["DFS", "обход в глубину", "поиск в глубину"],
 title: "DFS – обход в глубину",
 short:
 "Прёшь вперёд до упора, упёрся – откатываешься на
 formal:
 "Стек (или рекурсия). Из DFS растут: времена вход
 complexity: "O(V + E)",
 demo: "dfs",
 simulator: "stack-dfs",
},
{
 id: "binsect",
 labels: ["binsect", "бинсект", "бинарный поиск", "д
 title: "Бинарный поиск (и его самозванец binsect)",
 short:
 "Бинарный поиск НАХОДИТ готовый элемент в уже отс
 о не сортирует. «binsect» из конспекта – шутлив
 quicksort.",
 formal:
 "Поиск одного элемента в отсортированном массиве -
 O(n log n). Бинарный поиск сортировку не выполн
 complexity: "поиск: O(log n); сортировкой не являет
},
{
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "
 title: "Куча (priority queue)",
 short:
 "Не отсортированный список, а «мешок, из которого
 formal:
 "Полное бинарное дерево в массиве: дети узла i ле
 complexity: "вставка/извлечение O(log n), «посмотре
 demo: "heap",

```

```

 formal: "Из корня по символам; в вершине хранится ф.
 complexity: "поиск слова — $O(|\text{слова}|)$ ",
 demo: "trie",
 simulator: "aho-corasick",
},
{
 id: "bfs-on-trie",
 labels: [
 "обход дерева",
 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
],
 title: "Обход бора в ширину (тот самый «какой-то об
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффиксн
 о есть строго по уровням.",
 formal:
 "Кладём в очередь детей корня (их ссылка = корень
 кладём с в очередь.",
 complexity: " $O(\text{суммарной длины словаря} \times \text{алфавит})$ ",
 demo: "trie-bfs",
 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки",
 title: "Суффиксная ссылка",
 short:
 "«Куда откатиться, если следующая буква не подошл
 строки.",
 formal: " $\text{link}(v) = \text{go}(\text{link}(\text{parent}(v)), \text{ch})$. Считае
 demo: "suffix-link",

```

```

 id: "sparse-table",
 labels: [
 "разреженная таблица", "разреженная таблица", "sp",
 "разреженная", "двоичные подьёмы", "двоичный подь",
],
 title: "Разреженная таблица (метод двоичных подьёмов)",
 short:
 "Заранее считаем ответ для всех отрезков длины 1,
 м.",
 formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
 complexity: "препроцессинг $O(n \log n)$, запрос $O(1)$ ",
 demo: "sparse-table",
 simulator: "sparse-table",
},
{
 id: "segment-tree",
 labels: ["дерево отрезков", "дереве отрезков", "дер"],
 title: "Дерево отрезков",
 short:
 "Матрёшка из отрезков: корень отвечает за весь ма
 товых кусков.",
 formal: "Строится за $O(n)$, запрос и обновление — $O(\log n)$ ",
 demo: "segment-tree",
 simulator: "segment-trees",
},
{
 id: "bridge",
 labels: ["мост", "моста", "мосты", "мостов", "мостов"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается",
 formal: "Ребро (u,v) — мост $\iff \text{low}[v] > \text{tin}[u]$, где $\text{tin}[u]$ — время захода в вершину u при обходе в глубину (и $\text{low}[v]$ — минимальное tin среди всех вершин, достижимых из v, не проходя через u).",
 complexity: " $O(V + E)$ одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{

```

```

 labels: [
 "NP-полная", "NP-полнота", "NP-трудная", "NP-полнота",
 "класс P", "полином", "полиномиальное",
],
 title: "Класс NP и NP-полнота",
 short:
 "NP — «решение проверить легко, найти трудно» (суть проблемы в NP.",
 formal: "Задача NP-полна, если она в NP и к ней полнота",
 demo: "np",
},
{
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов", "потенциал", "потенциалы", "потенциалов"],
 title: "Потенциалы (перевзвешивание Джонсона)",
 short:
 "Трюк «поднять все дороги на одинаковую высоту»: для каждой вершины u и v положить $w'(u, v) = w(u, v) + h(u) - h(v)$, где h — потенциал. Тогда кратчайший путь сохраняется.",
 formal: " $w'(u, v) = w(u, v) + h(u) - h(v)$, где h — потенциал",
 demo: "relax",
 simulator: "johnson-algo",
},
{
 id: "scc",
 labels: ["компонента сильной связности", "компоненты сильной связности", "компоненты", "компоненты", "компоненты", "компоненты"],
 title: "Компоненты сильной связности",
 short:
 "Группы вершин, где из каждой можно дойти до каждой другой",
 formal: "Косарайю: DFS с сохранением порядка выхода из вершин",
 complexity: " $O(V + E)$ ",
 demo: "scc",
 simulator: "scc-kosaraju",
},
{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-функция", "префикс-функции",
 "префикс", "префикса", "суффиксом", "суффикс", "суффиксы", "суффиксов"
],

```

```

 demo: "dp",
 simulator: "dynamic-programming",
},
{
 id: "shortest-path",
 labels: ["кратчайший путь", "кратчайшие пути", "кра
 title: "Кратчайший путь",
 short:
 "Самый дешёвый маршрут из А в В. «Дешёвый» — по с
 formal:
 "Дейкстра — неотрицательные веса, $O(E \log V)$. Фор
 ы, $O(V^3)$.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикл
 title: "Отрицательный цикл",
 short:
 "Круг, пройдя по которому вы становитесь «богаче»
 formal:
 "Форд–Беллман: если на V -й итерации ещё удаётся с
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешёвое ребро", "самое дешёвое ребро", "са
],
 title: "Жадный алгоритм",
 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас
 formal:
 "Корректность обычно доказывается «леммой о разре
 demo: "mst",

```

```
{
 id: "adjacency",
 labels: ["матрица смежности", "список смежности", "список смежности"],
 title: "Способы хранить граф",
 short:
 "Матрица смежности — таблица «есть ли ребро», быстрая и экономно для разреженных графов.",
 formal: "Матрица: проверка $O(1)$, память $O(V^2)$. Список: проверка $O(V)$, память $O(V)$.",
},
{
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкстры"],
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший не раскрытый.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали минимальную кучу.",
 complexity: "O(E log V) с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "bellman-ford",
 labels: ["Форд-Беллман", "Форда-Беллмана", "Беллмана-Форда"],
 title: "Алгоритм Форда-Беллмана",
 short:
 "Тупой, но честный: V-1 раз проходим по ВСЕМ рёбрам.",
 formal:
 "После k-й итерации гарантированно верны все кратчайшие пути, если их нет.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
},
{
 id: "floyd",
```

```

 simulator: "mst-boruvka",
 },
 {
 id: "kruskal",
 labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
 title: "Алгоритм Краскала",
 short:
 "Сортируем все рёбра по весу и берём подряд те, ч
 formal:
 "Сортировка $O(E \log E)$ + E операций union/find. C
 complexity: " $O(E \log E)$ ",
 demo: "mst",
 simulator: "mst-kruskal",
 },
 {
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "
 "компоненте связности", "связности", "связный", "
],
 title: "Компоненты связности",
 short:
 "Граф-архипелаг: острова, между которыми нет мост
 бой.",
 formal:
 "Считаются одним проходом: пока есть непосещённая
 complexity: " $O(V + E)$ ",
 demo: "components",
 simulator: "graph-dfs-bfs",
 },
 {
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "гра
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:

```



```

 formal:
 "Без балансировки дерево может вырождаться в список",
 complexity: "O(высоты): от O(log n) до O(n)",
 demo: "segment-tree",
 },
 {
 id: "inclusion-exclusion",
 labels: [
 "включений-исключений", "включения-исключения", "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём большой и маленький, а лишние отрезали дважды.",
 formal:
 "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
 complexity: "запрос O(1) после O(nm) предподсчёта",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "reduction",
 labels: ["сведение", "сведения", "сведении", "сведённый"],
 title: "Сведение задач",
 short:
 "«Я не умею решать A, зато умею B». Если A можно свести к B, то B можно решить.",
 formal:
 "A ≤p B: есть полиномиальная функция, переводящая экземпляры A в экземпляры B, решая которые можно решить A.",
 demo: "np",
 },
 {
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива.",
 }
}

```

```

 канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию",
 demo: "zig",
 simulator: "splay-rotations",
 },
 {
 id: "zig-zig",
 labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
 title: "Zig-Zig – x и родитель с одной стороны",
 short:
 "Дерево вытянулось в «бамбук»: g → p → x идут в од
 м (p, x). Именно это вдвое сокращает глубину вс
 formal:
 "Если крутить наивно (два раза поднимать x), бамб
 demo: "zig-zig",
 simulator: "splay-rotations",
 },
 {
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag – x и родитель с разных сторон",
 short:
 "Узлы идут «змейкой»: p – левый сын g, а x – прав
 p и g становятся двумя детьми x.",
 formal: "Эквивалентно двойному повороту AVL-дерева",
 demo: "zig-zag",
 simulator: "splay-rotations",
 },
];

```

```

/** Быстрый доступ по id. */

```

```

export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

```

```

/** Все метки, отсортированные от длинных к коротким (ч

```

```

export const GLOSSARY_LABELS: { label: string; id: string }[] =
 GLOSSARY.map((e) => ({ label: e.labels[0], id: e.id }))
 .sort((a, b) => b.label.length - a.label.length);

```

```

 <h2 class="text-3xl font-bold text-white">Дейкстра
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в
 <div id="slot-chapter-image-dijkstra">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">
 охождение улицы), Дейкстра сломается, так как
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border">
 <summary class="p-4 font-bold text-slate-300">
```

```

 rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-2">
 еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman" class="w-100">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
 border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-2">
 г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
</div>
</section>`
 },
 {
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12">
 <div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Флойд
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">

```

```
category: "Строки",
content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 round
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">Осн
 <p class="text-lg font-mono text-white">Бор (Тр
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 у.</p>
 </div>

 <!-- Аналогии -->
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы движемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2 l
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
```

```

nodes[child].link = nodes[nodes[v].link].go

// Если суффиксная ссылка сама является сло
// Иначе наследуем terminal_link
if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].l
} else {
 nodes[child].term_link = nodes[nodes[ch
}

q.push(child);
 }
}
}

</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `
```

```

 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий
ого критерия. Это NP-полная задача, которую не
 },
 {
 question: "Куда ведёт эйлеров путь, если нечётных
options: [
 "Всё ещё можно пройти по всем рёбрам ровно раз",
 "Такого графа не существует",
 "Эйлерова пути нет – нужно удалять лишние рёбра",
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлер
ровно 2 нечётные или 0."
 }
],
"bridges-code": [
 {
 question: "После DFS у нас $low[u] = 4$, а $tin[v] =$
options: [
 "Да, так как $low[u] (4) > tin[v] (2)$ ",
 "Нет, $low[u]$ должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
 correctIndex: 0,
 explanation: "Условие моста: $low[u] > tin[v]$. В н
 },
 {
 question: "Для чего нужен массив $low[v]$?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого
 "Чтобы хранить глубину рекурсии"
],
 correctIndex: 1,
 explanation: " $low[v]$ хранит минимальное время вхо
поиска мостов и точек сочленения."
 },

```

```
 "Потому что 5 вершин — несчастливое число"
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, 3V - 6 = 9. 10 > 9"
 }
]
};
```

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 23/81: src/data/tickets/ticket1\_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМ

```
import { Chapter } from '../../types';

export const tickets1to5: Chapter[] = [
 {
 id: "segment-trees",
 title: "1. Дерево отрезков с операциями на отрезках",
 type: "html",
 description: "Дерево отрезков — структура данных для",
 category: "Оптимизации и Продвинутое",
 content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-violet-700">
 </div>

 <div class="space-y-8">
 <div id="intro" class="bg-slate-700/50 p-6 rounded">
 <h3 class="text-xl font-bold text-blue-400">
 </div>
 </div>
`
 }
];
```



```

 lazy[2 * node] += lazy[node];
 tree[2 * node] += lazy[node];
 lazy[2 * node + 1] += lazy[node];
 tree[2 * node + 1] += lazy[node];
 lazy[node] = 0;
 }
}
</pre>
</div>
</details>
</div>
</div>
</section>
`
},
{
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Сложность
 <h2 class="text-2xl sm:text-3xl font-bold text-white">2. Двумерная разреженная матрица (Sparse Table)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Задача
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Дана матрица ST размера $L \times R$ и количество запросов Q . Для каждого запроса (l, r, k) нужно найти значение в k -м элементе минимального покрывающего пути от l до r .
 <p class="text-xl font-mono text-white">Путь состоит из отрезков: $min(ST[L][k], ST[R - 2^k + 1][k])$.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">

```

```

<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 метода: Split и Merge.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Удар катаной (Split)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 по значению X. Все, кто меньше X улетают в
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Слияние капель (Merge)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 вого). Мы просто смотрим на корни: у кого Y
 как потомок.</p>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
```

```

 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: VIP-лифт
 </div>
 <p class="text-slate-300 text-sm mb
вится корнем дерева (VIP-статус). Если ты ч
ка почти не требуется времени!</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Балансировка чере
 </div>
 <p class="text-slate-300 text-sm mb
и становиться кривым как бамбук. Но как тол
 </p>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounde
// Zig, Zag
function rotate(x) {
 let p = x.parent;
 if (p.left === x) { // Right rotate
 p.left = x.right;
 x.right = p;
 } else { // Left rotate
 p.right = x.left;
 x.left = p;
 }
}

```

```

 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 × Аналогия 1: Вырезание прямоу
 </div>
 <p class="text-slate-300 text-sm mb
 з него маленький целевой прямоугольник в цел
 ый верхний угол отрезается дважды! Поэтому
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
];

```

```

 border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-1">
 дный и не умеет пересматривать уже "зафиксир
 </div>
 </div>
 <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "bellman-ford",
 title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="bellman-ford-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">15.
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-rose-400">
 <h3 class="text-xl font-bold text-rose-400">15.1. Алгоритм Форда-Беллмана
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана
 <p class="text-xl font-mono text-white">
 1. Инициализация
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-1">
 еряет ВСЕ возможные дороги V-1 раз подряд.

```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
 шаг class="bg-slate-900 text-pink-400" style="font-family: monospace;">k
 шу себе проходить через вершину k?></i>
 </p>
 <ul class="text-sm text-slate-300 space-y-4">
 Шаг k=0: Ты знаешь только
 Шаг k=1: Разрешаем пересадки
 е> через путь class="text-indigo-300" style="font-family: monospace;">k
 Шаг k=2: Разрешаем пересадки
 нутри этих путей уже могут быть пересадки ч
 ...
 Шаг k=N: Ты проверил все

 </div>
</div>
</section>`
 },
 {
 id: "johnson-algo",
 title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-amber-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-xl font-mono text-white">
 </div>
 </div>
 </div>
`
 }
]

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-emerald-300 italic"
 <p class="text-xl font-mono text-white"
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb"
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`
},
{
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-r
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-emerald-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"

```

```

<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-3 border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-6">
 изкому племени для объединения. К вечеру племена
 ства шлют гонцов к ближайшему чужому царству.
 </div>
 </div>
</div>
</section>`
 }
];

```

ФАЙЛ 25/81: src/data/tickets/ticket21\_24.ts  
 КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМЕР: 1.2 КБ

```

import { Chapter } from '../types';

export const tickets21to24: Chapter[] = [
 {
 id: "string-kmp",
 title: "21. Префикс-функция (π) — Взгляд назад (КМП)",
 type: "html",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Строки
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">21. Префикс-функция (π) — Взгляд назад (КМП)
 </div>
 <div class="space-y-8">

```



```
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
```

Пример вычисления (Скрыто)

```
</summary>
<div class="p-5 text-sm text-slate-300
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2
 a: Префиксов нет. <code>
 ab: Из начала ("a")
```

```
>
 abc: Опять мимо. <code>
 abca: 0! Начало ("a"
```

```
</code>
```

```
 abcab: Начало ("ab")
 /li>
```

```
 abcabc: Начало ("abc
 e>
```

```
 abcabcd: "d" всё исп
 "abca" vs "abcd"). Конец "abcd" не совпада
```

```

```

```
</div>
```

```
</details>
```

```
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
```

Код C++ (Скрыто)

```
</summary>
<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
```

```
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
```

```
</div>
```

```

</div>

<!-- Аналогия 2 -->
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Аналогия 2: LLM Самокопиаст
</div>
 <p class="text-slate-300 text-sm mb"
 В LLM Z-функция может применять
[i]</code> (где <i>i</i> указывает назад на
сама себя.
 </p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Пример вычисления (Скрыто)
</summary>
 <div class="p-5 text-sm text-slate-300"
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2"
 <code>Z[0]</code> – не счита
 <code>Z[1]</code> (bcabcd
[1] = 0</code>
 <code>Z[2]</code> (cabcd

 <code>Z[3]</code> (abcd
? Нет, в начале 'a', тут 'd'). Совпало 3 бу
 <code>Z[4], Z[5], Z[6]</code>

 </div>
</details>
<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

```
</div>
```

```
<!-- Аналогии -->
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
```

```
 <div class="bg-slate-800 p-6 rounded-lg border bo
```

```
 <div class="absolute -top-3 left-4 bg-slate-700
```

```
 emerald-500 shadow-md">
```

```
 Аналогия 1: Паутина (Бор)
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">Представ
```

```
 ет — мы падаем по суффиксной ссылке ("
```

```
 </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-2
```

```
 <div class="absolute -top-3 left-4 bg-rose-900
```

```
 -500 shadow-md">
```

```
 Аналогия 2: Матёшка (Терминальные)
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">Представ
```

```
 нашли и "he", потому что оно спрятано внутри
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border b
```

```
 <summary class="p-4 font-bold text-slate-400 ou
```

```
 r-slate-700/50">
```

```
 Код (Скрыто)
```

```
 </summary>
```

```
 <div class="p-5 text-sm text-slate-300 space-y-
```

```
 <pre class="bg-slate-950 p-4 rounded overfl
```

```
int get_link(int v) {
```

```
 if (t[v].link == -1) {
```

```
 if (v == 0 || t[v].p == 0) t[v].link = 0;
```

```
 else t[v].link = go(get_link(t[v].p), t[v].pch)
```

```
 }
```

```
 return t[v].link;
```

```
}
```

```

 <p class="text-slate-300 text-sm mb-4">
 пример сортировка чисел). Класс NP – это
 енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4
 rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ь отличный переводчик на английский (Сведения
 аче В, то В – это NP-Полная (сосредоточена
 </div>
 </div>
 </div>
</section>`
 }
};

```

ФАЙЛ 26/81: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление, DFS, BFS",
 type: "html",
 description: "Представление графов и базовые обходы",
 category: "Графы. База",
 content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">

```

```

<!-- Аналогия DFS -->
<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000080; border-radius: 10px; box-shadow: 5px 5px 0px #000080;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = Жадный алгоритм
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Что делает: Жадный алгоритм находит первую доступную вершину (например, минимальную по номеру) и пробует другой путь.
 </p>
 <ul class="text-xs text-indigo-300 list-disc list-inside">
 Где используется:
 Топологическая сортировка (линейное время)
 Поиск мостов и точек сочленения
 Сильно связанные компоненты Киркпатрика
 Поиск циклов и просто проверка связности

</div>

```

```

<!-- Аналогия BFS -->
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 ♂ BFS (Поиск в ширину) = Волна
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Что делает: Концентрические волны. Работает через Очередь (FIFO). Продвигается от начальной точки во все стороны.
 </p>
 <ul class="text-xs text-emerald-300 list-disc list-inside">
 Где используется:
 Кратчайший путь в Невзвешенном графе
 Алгоритм Ахо-Корасик (сборка слов)
 Алгоритм Форда-Фалкерсона / Поиск в ширину в графах
 Двунаправленный поиск для ускорения

</div>

```

```

</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Планарные. Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white">Эта теорема утверждает, что любой планарный граф можно раскрасить 4 цветами.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">Границы не пересекаются в одной точке по-
 этому можно раскрасить карты, чтобы страны не сливались цветами, Всегда можно всего 4
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {

```

```
description: "Разложение задач по порядку выполнения",
category: "Графы",
content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>
 </div>
 <div class="bg-slate-900/40 rounded-lg p-6">
 <summary class="p-4 font-bold text-slate-300">
```

```
description: "Разбиение орграфа на макро-вершины. А
category: "Графы. Продвинутые",
content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Компоненты
и <i>ориентированного</i> графа, в котором
 </div>
 </div>

 <p class="text-slate-300 text-sm mb-4">
 «Это просто цикл?» – Нет!
 </p>
 <ul class="list-disc list-inside text-sm text-slate-300">
 Если у тебя есть цикл <code>А → Б → А</code>
 Если ты добавишь вершину Г к
 четыре вершины – одна большая SCC.
 Правило: Если два цикла имеют хотя бы

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
 Аналогия: Интриги в офисе
 </div>
 <p class="text-slate-300 text-sm mb-4">
```



```

 </div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
 этот мост, экономикой обоих кусков придет
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {

```

```
<div class="absolute -top-3 left-4 bg-slate-900 border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Представь город, где два района соединены перекрестком (удаляют вершину) — районы будут хрупкостью системы.

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-emerald-500 shadow-md">
```

```
<div class="absolute -top-3 left-4 bg-emerald-500 border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У тебя несколько компьютеров в одной комнате (мостом). Сами свитчи — это точки соединения.

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
```

```
<summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p class="text-blue-400 font-bold">Особый случай
```

```
<p>
```

Для стартовой вершины обхода правил (она не может быть выше неё прыгать некуда). Поэтому для неё не нужно больше 1 независимого ребенка.

```
</p>
```

```
<div class="bg-slate-950 p-4 rounded-lg">
```

```
<pre class="text-xs font-mono text-slate-300">
```

```
void dfs(int v, int p = -1) {
```

```


 </div>
 </div>
 </section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Эйлеров цикл
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-indigo-400">Эйлеров цикл
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">Эйлеров цикл — это замкнутый путь,
 <p class="text-xl font-mono text-white">который проходит по каждому ребру графа
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрывая
 </div>
 <p class="text-slate-300 text-sm mb-4">Если граф имеет нечетное число вершин, то
 <p class="text-slate-300 text-sm mb-4">не существует замкнутого пути, который
 <p class="text-slate-300 text-sm mb-4">сходится нечетное число линий, значит, из каждой вершины
 <p class="text-slate-300 text-sm mb-4">а везде должно быть четное число (зашел-вышел)
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);
edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});

interface StepInfo {
 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
 tin: number[];
 low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;

```

```

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}
 v,
);

 if (currentLow[to] >= currentTin[v] && p !==
 ap.add(v);
 recordStep(
 `Найдена точка сочленения: вершина ${v}
 v,
);
 }
 }
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это
 v,
);
} else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он
 v,
);
}
};

for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
}

recordStep("Алгоритм завершен. Найдены все точки сочленения");
setSteps(newSteps);

```



```

 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-1
/>
<text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-events
>
 {node.label}
</text>

{isVisited && (
 <>
 <text
 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"

```

```

 <div className="w-3 h-3 rounded-full bg-rose-500">
 Текущая
 </div>
 <div className="flex items-center gap-2">
 <div className="w-8 h-1 bg-rose-500 border-1 border-rose-500 rounded">
 Мост
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 font-medium">
 ЛОГ ДЕЙСТВИЙ:
 </h5>
 <div className="space-y-2">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => {
 <div
 key={idx}
 className={`text-xs p-2 rounded ${
 idx === currentStepIndex ? 'bg-rose-500' : 'bg-slate-500'
 }`}
 >
 {step.desc}
 </div>
 })}
 </div>
 </div>

 <div className="mt-4 shrink-0">
 <div className="w-full bg-slate-800 h-2 rounded">
 <div
 className="bg-rose-500 h-full transition-all"
 style={{
 width: `${(currentStepIndex / Math.max(steps.length, 1)) * 100}%`
 }}
 />
 </div>
 </div>
 </div>
 </div>
</div>

```



```

 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
 { u: 'C', v: 'E', w: 4 },
 { u: 'D', v: 'F', w: 2 },
 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
};

const adjList = getAdjList(edges);

const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1 iterations' },
 { line: 4, text: ' for (u, v, weight) in E:' }
];

```

```

 log: `Итерация ${iter}: Проверяем ${edge.u} →
 updated ? ` Релаксация успешна! dist[${edge
 }`,
 activeCodeLine: updated ? 6 : 5,
 });
}

if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distance
 log: ` Ранний выход после итерации ${iter}.
 activeCodeLine: 10
 });
 break;
}
}

if (simulationSteps[simulationSteps.length - 1].ite
let cycleFound = false;
for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.
 distances: { ...dist }, previous: { ...prev
 log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${e
 activeCodeLine: 9,
 });
 break;
 }
}

if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev }
 log: ` Проверка завершена. Ни одно ребро не
 activeCodeLine: 10,
 });
}
}

```



```

 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y - 28} fill=
 right="bold" textAnchor="middle" className="
 <text x={node.x} y={node.y + 35} fill=
 split(' ')[0]}</text>
 </g>
);
 }}}
</svg>

<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap g
 -colors">
 <span className="font-bold text-white
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingE
 return (
 <span key={idx} className={`px-2
edge.w < 0 ? 'bg-rose-900/80 text-rose-300
 {edge.v} ({edge.w})

)
 })}
 </div>
)})}
</div>
</div>
</div>
</div>

```

```
 </div>
 </div>
);
 }
}
```

---

ФАЙЛ 29/81: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useCallback } from 'react';
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react';
```

```
const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [], 6: [],
};
```

```
interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
}
```

```

steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
});

const neighbors = ADJ[curr];
if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добав.
 });
 }
 }
} else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,

```

```

 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
 }, [isPlaying, stepIdx, handleNext]);

return (
 <div className="bg-slate-900 border-2 border-slate-100 p-4">
 <div className="flex flex-col md:flex-row items-center justify-between">
 <div>
 <h3 className="text-2xl font-bold text-white">Музыка
 Ал
 </h3>
 <p className="text-slate-400 text-sm mt-1">Видео
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded">
 <button
 onClick={() => { setIsPlaying(false); setStep(0); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-blue-300"
 title={isPlaying ? 'Пауза' : 'Воспроизведение'}
 >
 {isPlaying ? <Pause className="w-5 h-5" /> : <Play className="w-5 h-5" />}
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-emerald-300 disabled:opacity-50 disabled:cursor-not-allowed"
 title="Следующий шаг вперед"
 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>
);

```

```

 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }

 return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors durati
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold tex
 >
 {n.label}
 </text>
 </g>
);
 }
 }
 </svg>
</div>

```

```

 { /* Код и Очередь */ }
 <div className="flex flex-col gap-4">

```



```

 <span className="text-slate-500 text-x
) : {
 step.queue.map(({qId, i}) => {
 <span key={`${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
 {qId}

))
 })
</div>
<div className="mt-2 text-sm text-amber-300
ms-center justify-center">
 "{step.logs}"
</div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 30/81: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

```

```

const imageMap: Record<string, { src: string; alt: string;
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 caption: 'Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',

```

```

import { ChevronLeft, ChevronRight } from "lucide-react"
import type { Chapter } from "../types";

interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
}

/**
 * Переходы «предыдущая / следующая тема» — как на Code
 * Компактный вариант живёт в шапке главы, крупный — по
 */
export const ChapterNav: React.FC<Props> = ({ prev, next,
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disab
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>
 <span className="text-slate-500 font-mono tabular
 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тем
 className="flex items-center gap-1 px-2.5 py-

```

```
 </button>
)}
 </nav>
);
};
```

---

ФАЙЛ 32/81: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useEffect, useRef, useState } from 'react';
import { Check, Download, Loader2, Maximize2, MousePointer } from 'lucide-react';
import type { Chapter } from '../types';
import { useTermHints } from '../hooks/useTermHints';
import { TermPopover, type HintAnchor } from './hints/TermHint';
import { getViz } from './vizRegistry';
import { ChapterNav } from './ChapterNav';
import { downloadChapterHtml } from '../utils/exportHtml';
```

```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;
```

```
export const ChapterView: React.FC<Props> = ({ chapter, prev, next, index, total, onGo, onOpenSimulator }) => {
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");
```

```

 [cancelHide]
);

 const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-body');
 if (el) open(el);
 };

 const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-body');
 if (el) scheduleHide();
 };

 const handleClick = (e: React.MouseEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-body');
 if (!el) return;
 e.preventDefault();
 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
 };

 const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>('.chapter-body');
 if (el) open(el);
 };

 const viz = getViz(chapter.id);

 return (
 <>
 <article className="chapter-body">
 {/* панель загрузки: сохранить именно эту страницу */}
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 />
 </div>
 </article>
 </>
);
}

```

Подчёркнутые термины – интерактивные: наведи  
объяснение на пальцах, мини-анимацию и кнопки

</span>

</p>

{viz && {

<section id="chapter-viz" className="mt-8 scroll" />

<div className="flex items-center justify-between" />

<h3 className="flex items-center gap-2 text-indigo-600" />

<Sparkles className="w-4 h-4 text-indigo-600" />

Демонстрация: {viz.title}

</h3>

<button

onClick={() => onOpenSimulator(chapterIndex)} />

className="flex items-center gap-1.5 text-indigo-600" />

size-300 hover:text-white hover:border-indigo-600" />

>

<Maximize2 className="w-3.5 h-3.5" /> {viz.title} />

</button>

</div>

{viz.hint && <p className="text-xs text-slate-500" />

<viz.Component />

</section>

}}

<ChapterNav prev={prev} next={next} index={index} />

</article>

<TermPopover

anchor={anchor}

onClose={() => setAnchor(null)}

onOpenSimulator={(id) => {

setAnchor(null);

onOpenSimulator(id);

}}

onCardEnter={cancelHide}

onCardLeave={scheduleHide}

/>

```
];
```

```
interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}
```

```
const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];
```

```
const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];
```

```
const DFS_STEPS: DfsStep[] = [
 {
```

```

 log: "Видим соседа A (уже посещён)! Обратное ребро
 activeCodeLine: [7,8,9]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
 activeEdge: [2,3], backEdge: null,
 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
 activeEdge: [4,5], backEdge: null,
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).
 activeCodeLine: [7,8,9]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо
 activeCodeLine: [11,13,16,17]
},

```





```

 return <circle cx={n.x} cy={n.y} r={18} f
 >;
 }}())}
 {GRAPH_NODES.map(n => {
 const cur = step.currentNode === n.id;
 const vis = step.visitedIndices.includes(n.id);
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b" : "#fff"}
 stroke={cur ? "#fff" : st ? "#34d399" : "#000"}
 strokeWidth={2.5}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y+3} textAnchor="middle"
label}</text>
 {vis && (
 <div
 <rect x={n.x-18} y={n.y-28} width={36} height={10}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor="center">
 {l:{step.low[n.id]||"?"}}</text>
 </div>
)}
 </g>
)}}
 </svg>
 <div className="flex items-center justify-between" >
 Шаг {dfsIdx+1}/{DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-10" >
 <div className="bg-indigo-600 h-full transition-all" >
 </div>
 </div>
 </div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
 {/* Code panel */}

```

```

 </div>
 </div>

 { /* Log */ }
 <div className="bg-indigo-950/10 border borde
 <p className="text-[11px] text-slate-300 le
 <div className="mt-2 pt-2 border-t border-s
 Мосты:</
 <span className="font-mono font-bold text
 {step.bridges.length > 0 ? step.bridges

 </div>
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 34/81: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

```

```

 { line: 6, text: ' if d > dist[u]: continue'
 { line: 7, text: ' for v, weight in graph.ne
 { line: 8, text: ' if dist[u] + weight <
 { line: 9, text: ' dist[v] = dist[u]
 { line: 10, text: ' pq.push((dist[v]
 { line: 11, text: ' return dist // Все кратчайши
];

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999
 const visited: Record<string, boolean> = { A: false
 const prev: Record<string, string | null> = { A: nu

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пи
 activeCodeLine: 2,
 });

```

```

for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }

 if (!u) break;

 visited[u] = true;

```

```

 });

 setSteps(simulationSteps);
 }, []);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 {/* Шапка симулятора */}
 <div className="flex flex-wrap items-center justify-between"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Полн
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button
 onClick={() => { setIsPlaying(false); setCu

```

```

 <marker id="arrow-dh-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6"
 <path d="M0,0 L6,3 L0,6 z" fill="#10b98b" />
 </marker>
 </defs>
 { /* Рёбра */ }
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &&
 currentStep.checkingEdge.v === edge.v);

 const isOptimal = currentStep.previous[edge.id] === 'optimal';

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-active)';
 else if (isOptimal) marker = 'url(#arrow-dh-opt)';

 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b98b' : '#1e293b'}
 strokeWidth={isChecking ? 5 : isOptimal ? 2 : 1}
 markerEnd={marker}
 className="transition-all duration-300"
 strokeDasharray={isChecking ? '4,4' : isOptimal ? '1,1' : '0'}
 />
 <circle
 cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)}
 r="12" fill="#1e293b"
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b98b' : '#1e293b'}
 strokeWidth={isChecking ? 2 : isOptimal ? 1 : 0}
 />
 <text

```

```

 }}}
 </svg>
 <div className="w-full bg-slate-950/80 p-4 rounded-2xl">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
 </p>
 </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/50 p-4 rounded-2xl">
 <h4 className="text-lg font-bold text-emerald-400">
 <div className="space-y-2.5 flex-1 overflow-y-hidden">
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.currentNode === u
 return (
 <div key={u} className={`p-3 rounded-lg ${
 isCurrentNode ? 'bg-emerald-900/60 text-slate-300' : 'bg-slate-950/80 text-slate-300'
 }`} >
 <div className="flex items-center gap-2">

 {u}

 →
 </div>
 <div className="flex flex-wrap gap-2">
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep.currentNode === edge.v
 return (
 <span key={idx} className={`px-2 py-0.5 rounded ${
 isCheckingThis ? 'bg-amber-900/60 text-slate-300' : 'bg-slate-950/80 text-slate-300'
 }`} >
 {edge.v}

 →

 {edge.w}

)
 })}
 </div>
 </div>
)
 })}
 </div>

```

```

 </td>
 <td className="py-2.5 px-3 font
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text
 {prev || '-'}
 </td>
 </tr>
);
 }}}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg p
 {codeLines.map((item) => {
 const isActive = currentStep.activeCode
 return (
 <div key={item.line} className={`py-1
 isActive ? 'bg-indigo-900/80 text
 }`}>
 <span className="text-slate-600 sel
 <span className="whitespace-pre fle
 {isActive && <span className="text-
 }
 </div>
)};
 }}}
 </div>
 </div>
</div>
</div>
</div>

```

```

useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibМемо(${n}). Проверяем наличие в
codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано
codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
 }

 if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвр
codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
 }
 }

```



```

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
 }, [targetN]));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

return (
 <div className="bg-slate-900 rounded-2xl border border-slate-800" >
 <div className="flex flex-col lg:flex-row lg:items-center" >
 <div>
 <div className="flex items-center gap-3 mb-2" >
 <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text-white" >
 </h3>
 <p className="text-sm text-slate-400" >
 Смотри, как таблица DP кэширует результаты
 </p>
 </div>
 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center gap-2 bg-slate-800" >

```

```

 {isPlaying ? <Pause className="w-4 h-4" />
 {isPlaying ? 'Пауза' : 'Пуск'}}
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === steps.length
 className="p-2 text-slate-400 hover:text-
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
 </div>
</div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded
 <h4 className="text-xs font-bold uppercase trac
 <div className="flex flex-wrap items-center gap
 {Array.from({ length: targetN }, (_, i) => i
 const isCached = num in memoState || num <=
 const val = num <= 2 ? 1 : memoState[num];
 const isCurrent = currentStep?.n === num;

 return {
 <div
 key={num}
 className={`flex flex-col items-center p
 isCurrent
 ? 'bg-emerald-950 border-emerald-500
 : isCached
 ? 'bg-slate-900 border-emerald-500/4
 : 'bg-slate-900/50 border-slate-800
 }`}
 >
 <span className="text-xs text-slate-400
 <span className={`text-xl md:text-2xl font
 {isCached ? val : '0'}
```

```

 : 'border-transparent text-slate-400 hover:
 }` }
 >
 <Code className="w-4 h-4" />
 Интерактивный код
</button>
</div>

```

```

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify-between" >
 {activeTab === 'table' && (
 <div className="bg-slate-950 p-6 rounded-2xl border" >
 <h5 className="font-bold text-white text-sm" >

 </h5>
 <p className="text-sm text-slate-400" >
 В классической рекурсии для $N=\{targetN\}$ по
 вычисленное значение в таблице, мы момента
 </p>
 <div className="p-4 bg-slate-900 rounded-xl border" >
 Всего шагов симуляции: {steps.length}

 </div>
 </div>
)}

```

```

{activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl border" >
 <div className="bg-slate-900 px-6 py-3 border" >

 </div>
 <pre className="p-6 font-mono text-sm space" >
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine === lineNum;
 return (

```

---

ФАЙЛ 36/81: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState } from "react";
import { Undo } from "lucide-react";

const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
 { u: 0, v: 1 }, { u: 0, v: 2 },
 { u: 1, v: 3 }, { u: 2, v: 4 },
 { u: 1, v: 2 },
 { u: 3, v: 4 }
];

export function EulerSimulator() {
 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Кликни на вершину");
 const [c1Done, setC1Done] = useState(false);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
```

```

<h4 className="text-base font-bold text-white">
<button onClick={resetC1} className="flex items
 old border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> C6poc
</button>
</div>

<div className="bg-slate-950 rounded-xl p-4 borde
 -hidden">
<div className="absolute inset-0 bg-[radial-gra
<svg className="w-full h-[260px] z-10 relative"
 {C1_EDGES.map((e, i) => {
 const u = C1_NODES.find(n => n.id === e.u)!
 const v = C1_NODES.find(n => n.id === e.v)!
 const key = `${Math.min(e.u,e.v)}-${Math.ma
 const used = c1VisitedEdges.includes(key);
 return <line key={i} x1={u.x} y1={u.y} x2={
 stroke={used ? "#6366f1" : "#475569"}
 strokeWidth={used ? 4 : 2}
 className="transition-all duration-300" />
 }}}
 {C1_NODES.map(n => {
 const isLast = c1Path[c1Path.length-1] === n
 const visited = c1Path.includes(n.id);
 return <g key={n.id} className="cursor-poin
 {isLast && <circle cx={n.x} cy={n.y} r={1
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#
 stroke={isLast ? "#fff" : visited ? "#6
 strokeWidth={2.5}
 className="transition-all duration-200
 <text x={n.x} y={n.y+4} textAnchor="middl
 bel}</text>
 </g>;
 }}}
</svg>
<div className="absolute top-2 left-2 bg-slate-
 Pe6ëp: {c1VisitedEdges.length}/{C1_EDGES.leng

```

```

 options: [
 'Мы радуемся и добавляем его в минимальное остовное дерево',
 'Мы перекрашиваем их в синий цвет и делим граф пополам',
 'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
 'Мы вызываем алгоритм Дейкстры для поиска нового ребра',
],
 correctAnswer: 2,
 explanation: 'Верно! Если вершины уже в одном клане, добавление ребра
 Это нарушит структуру дерева.',
},
{
 id: 2,
 question: 'Почему алгоритм Дейкстры впадает в ступор?',
 options: [
 'Потому что он написан только для положительных весов',
 'Он жадный и считает, что каждый следующий шаг приведет к оптимальному решению',
 'Отрицательные ребра создают гравитационный коллапс',
 'Дейкстра просто не любил отрицательные балансы на счетах',
],
 correctAnswer: 1,
 explanation: 'Именно! Дейкстра уверен: если ты приедешь к дереву,
 ля чего нужен алгоритм Беллмана-Форда.',
},
{
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борувки?',
 options: [
 'Он использует меньше памяти благодаря коммунистическим идеалам',
 'Он умеет работать с отрицательными весами без циклов',
 'Он позволяет выполнять шаги слияния колхозов (коммуны)',
 'Он был разработан в Токио и поэтому самый быстрый',
],
 correctAnswer: 2,
 explanation: 'Абсолютно верно! В Борувке каждая коммуна должна
 делить вычисления на GPU или многопроцессорных кластерах',
},
{

```

```

 setIsAnswered(true);
 if (index === currentQ.correctAnswer) {
 setScore(prev => prev + 1);
 }
 };

const handleNext = () => {
 if (currentQIndex + 1 < quizQuestions.length) {
 setCurrentQIndex(prev => prev + 1);
 setSelectedOption(null);
 setIsAnswered(false);
 } else {
 setShowResults(true);
 }
};

const handleRestart = () => {
 setCurrentQIndex(0);
 setSelectedOption(null);
 setIsAnswered(false);
 setScore(0);
 setShowResults(false);
};

if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-800 p-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-4">Результаты
 <p className="text-slate-400 text-sm mb-6">Ты набрал {score} из {total} баллов
 </p>
 <div className="bg-slate-950 border border-slate-800 p-6">
 <p className="text-slate-400 text-sm uppercase">Правильные ответы
 <p className="text-5xl font-black text-indigo-400">100%

```

```

 {currentQ.question}
 </h4>

 <div className="space-y-4">
 {currentQ.options.map((option, idx) => {
 const isSelected = selectedOption === idx;
 const isCorrect = idx === currentQ.correctAnswer

 let optionStyle = "bg-slate-800/80 hover:bg-slate-800/90"
 if (isAnswered) {
 if (isCorrect) {
 optionStyle = "bg-emerald-950/80 border-emerald-500"
 } else if (isSelected) {
 optionStyle = "bg-rose-950/80 border-rose-500"
 } else {
 optionStyle = "bg-slate-800/40 border-slate-700"
 }
 }

 return (
 <div
 key={idx}
 onClick={() => handleSelectOption(idx)}
 className={"flex items-start gap-4 p-4 rounded-lg"}
 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 text-emerald-500" />
) : isAnswered && isSelected && !isCorrect ? (
 <XCircle className="w-6 h-6 text-rose-500" />
) : (
 <div className={"w-6 h-6 rounded-full " +
 "border-indigo-500 bg-indigo-500 text-white"}
 {String.fromCharCode(65 + idx)}
 </div>
)}
 </div>
 <div>
 {option}
 </div>
 </div>
)
 })}
 </div>
 <div className="flex-1 text-sm md:text-base">

```



ФАЙЛ 38/81: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
```

```
interface Step {
 k: number; i: number; j: number;
 matrix: number[][]; updated: boolean;
 log: string; activeCodeLine: number;
}
```

```
export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const [isPlaying, setIsPlaying] = useState(false);
```

```
 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];
```

```
 const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
```

```

 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]), update: true,
 log: `    Улучшение пути [${i}]→[${j}] через [${k}].`,
 activeCodeLine: 7,
 });
 }
 }
 }

 simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]), update: true,
 log: `    Все пары отработаны. Алгоритм Флойда завершён.`,
 activeCodeLine: 8,
 });

 setSteps(simulationSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
}, [isPlaying, currentStepIndex, steps]);

```

</div>

```
<svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-fw-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
 th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
 </defs>
 {Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id
 const vNode = nodesSvg.find((n) => n.id
 const isTarget = currentStep.i === u &&
 const isVia = (currentStep.i === u && c
 let strokeColor = '#475569'; let marker
 if (isTarget) { strokeColor = '#10b981'
 else if (isVia) { strokeColor = '#818cf8
 return (
 <g key={` ${u} - ${edge.v} `}>
 <line x1={uNode.x} y1={uNode.y} x2=
 markerEnd={marker} strokeDasharray={isTarg
 <circle cx={({uNode.x + vNode.x) / 2
 isVia ? '#818cf8' : '#64748b'} strokeWidth
 <text x={({uNode.x + vNode.x) / 2} y=
 '8fafc'} fontSize="10" fontWeight="bold" tex
 </g>
);
 });
 });
 })}
 {nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b'
 let strokeDasharray = isK ? [5, 5] : isI ? [5, 5] : isJ ? [5, 5] : [5, 5];
 let strokeColor = isK ? '#10b981' : isI ? '#10b981' : isJ ? '#10b981' : '#475569';
 let marker = isK ? 'arrow-fw-normal' : isI ? 'arrow-fw-normal' : isJ ? 'arrow-fw-normal' : 'arrow-fw-normal';
 let markerEnd = isK ? 'arrow-fw-normal' : isI ? 'arrow-fw-normal' : isJ ? 'arrow-fw-normal' : 'arrow-fw-normal';
 return (
 <g key={` ${node.id} `}>
 <circle cx={node.x} cy={node.y} r={10} fill={fill} stroke={stroke} strokeDasharray={strokeDasharray} />
 <text x={node.x} y={node.y} text={node.id} font-size="10" font-weight="bold" />
 </g>
);
 })}
```



```
import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};
```

```
type Action = {
 type: 'visit' | 'process' | 'backtrack';
```

```

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начало обхода из ${start}, ${vis.size} вершин в очереди`, visitedNodes: [...Array.from(vis)] });

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Обработка вершины ${v}`, visitedNodes: [...Array.from(vis)] });
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, desc: `Посещение вершины ${u}`, visitedNodes: [...Array.from(vis)] });
 }
 }
 }

 steps.push({ type: 'backtrack', node: '', desc: `Завершение обхода`, visitedNodes: [...Array.from(vis)] });

 return steps;
 }
}

```

```

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);

 const steps = useMemo(() => mode === 'dfs' ? generateDfsSteps() : generateBfsSteps(), [mode]);

 useEffect(() => {
 setStepIdx(0);
 }, [steps]);
}

```

```

<div className="flex flex-col md:flex-row gap-4" style={{ flex: 1 }}>
 {/* SVG Graph View */}
 <div className="bg-slate-900 border border-slate-800 p-4 flex flex-grow-1 min-h-[300px]">
 <svg className="w-full h-full max-w-[1000px] max-h-[600px]">
 {/* Edges */}
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);
 // Check if edge is part of current path
 const edgeTraversed = isVisited(e.u, e.v, currentPath);

 return (
 <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 className={edgeTraversed ? 'stroke-slate-500' : 'stroke-emerald-500'} />
);
 })}

 {/* Nodes */}
 {nodes.map(n => {
 const visited = isVisited(n.id, currentPath);
 const current = isCurrent(n.id, currentPath);

 let fillColor = "fill-slate-500";
 let strokeColor = "stroke-slate-500";
 let textColor = "fill-slate-500";

 if (visited) {
 fillColor = mode === 'dark' ? 'fill-white' : 'fill-slate-800';
 strokeColor = mode === 'dark' ? 'stroke-white' : 'stroke-slate-800';
 textColor = "fill-white";
 }

 if (current && step.type !== 'edge') {
 strokeColor = "stroke-amber-500";
 textColor = "fill-amber-500";
 }

 return (
 <div
 key={n.id}
 style={{
 position: 'relative',
 width: 40px,
 height: 40px,
 margin: 10px,
 display: 'flex',
 align-items: 'center',
 justify-content: 'center',
 }}
 >
 <div
 style={{
 width: 100%,
 height: 100%,
 background: fillColor,
 border: 2px solid strokeColor,
 border-radius: 18px,
 }}
 />
 <div style="font-size: 10px; color: textColor; margin-top: 5px;">{n.id}
 </div>
);
 })}
 </svg>
 </div>
 <div style="flex: 1; padding: 10px; border: 1px solid #ccc; border-radius: 10px; background-color: #f9f9f9; margin-top: 10px;">
 <div style="border-bottom: 1px solid #ccc; padding-bottom: 10px; margin-bottom: 10px;">
 <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 10px;">
 <div style="font-weight: bold; font-size: 1.2em;">Path Information
 <div style="font-size: 0.8em; color: #666;">
 Step: {stepIndex} / {steps.length} | Type: {step.type} | Label: {step.label}
 </div>
 </div>
 <div style="font-family: monospace; font-size: 0.9em;">
 Path: {currentPath.join(' > ')}
 </div>
 </div>
 <div style="padding-top: 10px;">
 <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 10px;">
 <div style="font-weight: bold; font-size: 1.2em;">Next Steps
 <div style="font-size: 0.8em; color: #666;">
 Available: {availableSteps.length} | Selected: {selectedSteps.length}
 </div>
 </div>
 <div style="font-family: monospace; font-size: 0.9em;">
 Available: {availableSteps.map(s => s.label).join(', ')}
 Selected: {selectedSteps.map(s => s.label).join(', ')}
 </div>
 </div>
 </div>
</div>

```

```

0_10px_rgba(245,158,11,0.2)]]' : ''}
 `}>
 Узел: {item}
 {idx === (mode === 'dfs'
 <span className="ml-1
)}
 </div>
)}
 </div>
 {(!step.queueOrStack || step.queue
 <div className="text-slate-500
)}
</div>
</div>
</div>

<div className="flex flex-col sm:flex-row gap-4
 {/* Controls */}
 <div className="flex bg-slate-900 border
 <button onClick={() => {setAutoPlay
 "p-2 text-slate-400 hover:text-white hover:
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={() => setAutoPlay(
 justify-center min-w-[80px] ${mode === 'df
 >
 {autoPlay ? <><Pause size={16}
 </button>
 <button onClick={() => {setAutoPlay
 = steps.length - 1} className="p-2 text-sla
 hover:bg-transparent">
 <SkipForward strokeWidth={3} si
 </button>
 <button onClick={() => {setAutoPlay
 g-slate-800 rounded-lg ml-2 border-l border
 <RefreshCw strokeWidth={3} size
 </button>
 </div>

```



```

 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
 return a;
}

function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
 return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
 return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

```

```

const addLog = (msg: string) => setLog(prev => [msg,

// --- INSERT: Новый пациент поступает в приемный пок
const insertPatient = useCallback(({name: string, prio
 if (busy || heap.length >= 15) {
 addLog('⚠ Все палаты переполнены! Дождитесь выпис
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);

const preset = PATIENT_PRESETS[Math.floor(Math.rand
const finalName = name || preset.name.split(' ')[0]
const finalEmoji = preset.emoji;
const finalSymptom = preset.symptom;

const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
};

const nextHeap = heapInsert(heap, newPatient).map(x
// Mark specifically this new patient as 'in' in th
const target = nextHeap.find(x => x.priority === pr
const marked = nextHeap.map(x => x.id === (target?.

setHeap(marked);
addLog(` Поступил пациент: ${finalName} с тяжестью

setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);

```

```

 setTimeout(() => {
 // Run binary heap extraction
 setHeap(prev => {
 if (prev.length === 0) return [];
 const a = prev.map(x => ({ ...x }));
 a[0] = { ...a[a.length - 1] };
 a.pop();
 return siftDown(a).map(x => ({ ...x, animStat
 }));

 // Patient gets healed in the bed
 setTimeout(() => {
 setHealing(prev => prev ? { ...prev, animStat
 addLog(` Успех! Пациент ${topPatient.name} п
 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
 }, 650);
 }, [busy, heap]);

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat
 setLog([' База данных скорой помощи перезагружена.
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
});

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < heap.length) {

```

```

{/* УПРАВЛЕНИЕ */}
<div className="flex flex-col lg:flex-row items-s

 {/* Добавить случайного */}
 <button
 onClick={handleRandomInsert}
 disabled={busy || heap.length >= 15}
 className="flex-1 min-w-[150px] bg-gradient-to
 opacity-40 disabled:cursor-not-allowed text
 ve:scale-95 transition-all cursor-pointer f
 >
 Привезти по Скорой (INSERT)
 </button>

 {/* Добавить кастомного */}
 <form onSubmit={handleCustomInsert} className="
 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border borde
 er-emerald-500 transition-colors placeholde
 />
 <div className="flex flex-col items-center mi
 <span className="text-[9px] font-mono text-
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e
 className="w-20 accent-emerald-500 cursor
 />
 </div>
 <button

```

```

<div className="relative w-full h-full min-h-
 <svg className="absolute inset-0 w-full h-f
 {edges}
 </svg>

 {heap.map((patient, idx) => {
 const pos = nodePos(idx);
 const isRoot = idx === 0;
 return (
 <div
 key={patient.id}
 className={`
 absolute flex flex-col items-center
 -translate-x-1/2 -translate-y-1/2 t
 ${patient.animState === 'in' ? 'anim
 ${patient.animState === 'winner' ?
 ${patient.animState === 'out' ? 'an
 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mon
 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400">

```

```

 <div className="text-center text-slate-600">

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Ждём са
 </p>
 </div>
 </div>
 </div>
 </div>

 <div className="w-full h-3 bg-gradient-to-r f
 </div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-emerald-!
 {entry}
 </div>
 </div>
 </div>
</div>
);
};

```

ФАЙЛ 41/81: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { createContext, useContext, useEffect, u
```

```

done: "#10b981",
warn: "#f59e0b",
bad: "#f43f5e",
edge: "#475569",
};

export const Svg: React.FC<{ children: React.ReactNode;
const paused = useContext(DemoPauseContext);
return (
 <div className="w-full">
 <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
 {children}
 </svg>
 {caption && (
 <div className="text-[10px] text-slate-400 text-center">
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center">
 {paused ? "пауза – курсор на карточке" : "наведи"}
 </div>
 </div>
);
};

```

```

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}>
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </circle>
 </g>
);

```

```

const viaOpen = step >= 1;
const relaxed = step >= 2;
return (
 <Svg
 caption={
 step === 0
 ? "k ещё запрещена: путь $i \rightarrow j = 9$ "
 : step === 1
 ? "Разрешаем пересадку через k: $3 + 4 = 7$ "
 : " $7 < 9 \rightarrow D[i][j] = 7$ "
 }
 >
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
 </Svg>
);
};

```

/\*\* Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"],
 ["D", "E"],

```



```

 : "Вес ребра – цена перехода"
 }
>
 {edges.map(([u, v, w], i) => {
 const a = pos[u];
 const b = pos[v];
 return (
 <g key={i}>
 <Edge a={a} b={b} color={step >= 1 ? C.active : C.idle}
 {step >= 2 && (
 <text
 x={({a[0] + b[0]) / 2}
 y={({a[1] + b[1]) / 2 - 4}
 textAnchor="middle"
 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 >
 {w}
 </text>
)}
 </g>
);
)}
);
 </Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

```

```

)))
 </Svg>
);
};

/* ----- Splay: три случая поворота -----

type Pt = [number, number];

const SplayFrames: React.FC<{
 frames: { pos: Record<string, Pt>; edges: [string, string][];
 captions: string[];
 ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);
 const f = frames[step];
 const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#f1c40f";

 return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idle} />
 <text textAnchor="middle" dominantBaseline="middle">
 {id}
 </text>
 </g>
))}
 </Svg>
);
};

```

```

 }}
 />
};

```

ФАЙЛ 43/81: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
 const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step,
 {GRAPH_EDGES.map(([u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
 }}})
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
 }}})
 </Svg>
);
};

```

```

 return (
 <g key={n.id}>
 <rect x={n.x - 34} y={n.y - 12} width={68} h
 " }} />
 <text x={n.x} y={n.y} textAnchor="middle" d
 {idx >= 0 && <circle cx={n.x + 30} cy={n.y
 {idx >= 0 && {
 <text x={n.x + 30} y={n.y - 12} textAncho
 x + 1}</text>
 }}
 </g>
);
 }}}
 <text x={224} y={62} textAnchor="middle" fontSize
 <text x={224} y={74} textAnchor="middle" fontSize
 </Svg>
);
};

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Пут
 <Edge a={[60, 70]} b={[200, 70]} color={improved
 <text x={130} y={54} textAnchor="middle" fontSize
 <Node x={60} y={70} r={20} fill={C.active} label=
 <Node x={200} y={70} r={20} fill={improved ? C.do
 <text x={60} y={106} textAnchor="middle" fontSize
 <text x={200} y={106} textAnchor="middle" fontSiz
 <text x={130} y={22} textAnchor="middle" fontSize
 </Svg>
);
};

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);

```

```

 })
 </Svg>
);
};

export const MstDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10]
 };
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц`
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
 <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
)}
)}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
 </Svg>
);
};

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["B",

```

```

 });
 };

export const StackDemo: React.FC = () => {
 const states = [["A"], ["A", "B"], ["A", "B", "C"], [
 const step = useLoop(states.length, 800);
 const items = states[step];
 const popping = step >= 3;
 return (
 <Svg caption={popping ? "pop: забираем ВЕРХНИЮЮ тара."
 <rect x={90} y={22} width={80} height={96} rx={6}
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={
 fill={i === items.length - 1 ? (popping ? C
 <text x={130} y={111 - i * 26} textAnchor="mi
 </g>
))}
 <text x={130} y={16} textAnchor="middle" fontSize=
 </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeigh
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХ
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХ
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>

```

```
export const TrieDemo: React.FC = () => {
 const step = useLoop(TRIE_NODES.length + 1, 700);
 return (
 <Svg caption="Путь от корня по буквам = слово; общий"
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]]}
 {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x}
 </Svg>
);
};
```

```
export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? levels[step - 1] : [];
 return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, M
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]]}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch}
 fill={wave.includes(n.id) ? C.active : doneId
)}}
 <text x={6} y={12} fontSize={9} fill={C.dim}>очер
 </Svg>
);
};
```

```
export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3
 const step = useLoop(links.length + 1, 900);
 return (
```

```

 </g>
)})
 </Svg>
);
};

export const PrefixSumDemo: React.FC = () => {
 const a = [3, 1, 4, 1, 5, 9];
 const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
 const step = useLoop(4, 1200);
 const l = 1, r = 3;
 const show = step >= 1;
 return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -`
 : `о префиксные суммы P`} >
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.dim} />
 <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i === r ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: "fill 0.2s" }} />
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
))}
 </Svg>
);
};

```



```

const step = useLoop(2, 1400);
return (
 <Svg caption={step === 0 ? "Найти решение – долго ("}
 <ellipse cx={130} cy={64} rx={112} ry={48} fill="white" />
 <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333" />
 <text x={92} y={64} textAnchor="middle" dominantBaseline="middle" />
 <text x={206} y={34} textAnchor="middle" fontSize={14} />
 <circle cx={196} cy={84} r={13} fill={step === 1 ? "red" : "white"} />
 <text x={196} y={84} textAnchor="middle" dominantBaseline="middle" />
 <text x={130} y={124} textAnchor="middle" fontSize={14} />
 </Svg>
);
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс` : ""}
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={20}
 fill={step > 0 && (idx < len || (idx > i - len)} ? "red" : "white"}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle" />
 <text x={32 + idx * 33} y={70} textAnchor="middle" />
 <text x={32 + idx * 33} y={100} textAnchor="middle" />
 <text x={32 + idx * 33} y={130} textAnchor="middle" />
 </g>
))}
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={14} />
 π[i] – длина самого длинного «префикс = суффикс»
);
};

```

```
import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
import {
 AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
 SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
 ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBa
 ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components" |
 | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
```

```
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";

interface Props {
 vizId: string | null;
 onClose: () => void;
}

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
 <div className="fixed inset-0 z-[110] flex items-end
 <div className="absolute inset-0 bg-slate-950/80
 <div className="relative w-full sm:max-w-5xl max-h-
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py-
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm
 {viz.hint && <p className="text-[11px] text
 </div>
```

```

 onOpenSimulator: (vizId: string) => void;
 onCardEnter: () => void;
 onCardLeave: () => void;
 }

const CARD_W = 320;
const GAP = 10;

export const TermPopover: React.FC<Props> = ({ anchor, ...props }) => {
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number }>({ top: 0, left: 0 });

 useEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

 setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left });
 }, [anchor]);

 useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 }, []);

```

```

<div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
 <p className="text-[13px] leading-relaxed text-gray-600">
 {entry.demo} && <MiniDemo kind={entry.demo} />

 {entry.formal && (
 <p className="text-[11.5px] leading-relaxed text-gray-600">
 {entry.formal}
 </p>
)}

 {entry.complexity && (
 <div className="text-[11px] font-mono text-gray-600">
 Сложность: {entry.complexity}
 </div>
)}

 {viz && simulatorId && (
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center text-gray-600 font-medium rounded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
)}
</div>
</div>,
document.body
);
};

```

```

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-width-2px";
 if (step === 2) return "stroke-pink-500 stroke-width-2px";
 return "stroke-slate-700 stroke-dasharray-4 4 stroke-width-2px";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-width-2px";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-width-2px";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-width-2px";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // отсюда и далее
 return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

```

```

 if (e.id === 'CA' && step > 6) return true;
 return false;
 };

 const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соед
ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгор
ных вершин:
 <span className="text-pink-300 ml-1
 </div>
);
 case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый в
 <div className="text-center font-mo
 Это повышает или понижает вес ребра
 </div>
);
 case 4: return (
 <div>

```

```

<div className="flex flex-col md:flex-row gap-4" style={
 <div className="bg-[#0f172a] flex-1 min-h-[300px] p-4">
 <svg width="400" height="300" class="h-full w-full">
 <defs>
 <marker id="arrow-slate" viewBox="0 0 10 5" orient="vertical"
start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: slategray; stroke-width: 2px;" />
 </marker>
 <marker id="arrow-slate-dim" viewBox="0 0 10 5" orient="vertical"
uto-start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: slategray; stroke-width: 2px; opacity: 0.5;" />
 </marker>
 <marker id="arrow-amber" viewBox="0 0 10 5" orient="vertical"
start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #f1c40f; stroke-width: 2px;" />
 </marker>
 <marker id="arrow-pink" viewBox="0 0 10 5" orient="vertical"
tart-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #f08080; stroke-width: 2px;" />
 </marker>
 <marker id="arrow-emerald" viewBox="0 0 10 5" orient="vertical"
o-start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #27ae60; stroke-width: 2px;" />
 </marker>
 </defs>

 {/* Edges */}
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) {
 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

```



```

? 'fill-[#2
: 'fill-sla
/>
<text x={n.x} y={n.
-white">
 {n.label}
</text>
 {pot !== null && {
 <g transform={`
fade-in zoom-in duration-500">
 <rect x="-1
k-500 stroke-1" />
 <text x="0"
font-bold fill-pink-400">
 {pot}
 </text>
 </g>
)}
 </g>
)
 }}}
</svg>
</div>

<div className="w-full md:w-[320px] fle
 {/* Log Box */}
 <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
 {getDesc()}
 </div>

 {/* Controls */}
 <div className="flex bg-slate-900 b
 <button onClick={() => setStep(
 className="p-3 bg-slate-800
-slate-400 transition-colors" title="C6pocw
 <RotateCw strokeWidth={3}

```

```

 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";

interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;

```

```

 : [...initialEdges],
 });
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый шаг",
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Записываем в порядок`,
 u,
);
};

```

```

 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека`,
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в`,
 u,
 true,
 sccId,
);
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент`,
 null,

```

```

const getSccColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slate
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
 </div>
)

```

```

 </marker>
 </defs>

 { /* Edges */ }
 { initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id === edge.u);
 const v = initialNodes.find((n) => n.id === edge.v);

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
 let stroke = "#475569";
 let markerId = "arrow";

 if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
 } else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
 }
 })

 // Adjust slightly for bidirectional link
 const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);

```

```

 cy={node.y}
 r="20"
 className={`transition-all duration
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
)}
 </g>
);
 }}}
</svg>

{step.transposed && (
 <div className="absolute top-4 left-4 bg-in
 1 rounded">

```

```

 </div>
 </div>

 {/* List of actions log */}
 <div>

 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]">
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
)}}
 </div>
 </div>
 </div>

 <div className="mt-4 pt-3 border-t border-slate-200">

 № {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx === steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-800 text-white px-2 py-1 rounded"
 >
 Далее
 </button>
 </div>
 </div>
 </div>
 </div>
</div>

```



```

 color: string;
}

interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

```

```
// Step 3: Inspect D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветны
 type: 'info'
 }, ...prev]));
},
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цв
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и
 type: 'info'
 }, ...prev]));
},
// Step 6: Include B-C
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный ц
 type: 'success'
 }, ...prev]));
},
}
```

```
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину I
 type: 'info'
 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину I
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.'
 type: 'success'
 }, ...prev]));
},
},
```

```

 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. С
 type: 'warning'
 }, ...prev]));
 },
 // Step 20: Reject E-H
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
 },
 // Step 21: Inspect H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю
 type: 'info'
 }, ...prev]));
 },
 // Step 22: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фи
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
 },
];

```

```

 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону верш
 type: 'info'
 }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в куч
 type: 'success'
 }, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к вершн
 type: 'info'
 }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-
 type: 'success'

```

```

 description: 'Ребро В-Е отброшено. Плесень не р
 type: 'error'
 }, ...prev]);
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (Е) тянет
 type: 'info'
 }, ...prev]);
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]);
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]);
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {

```

```

setVertices(prev => prev.map(v => v.id === 7 ? {
setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
}, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.',
 type: 'warning'
 }, ...prev]));
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
},
// Step 22: Pop H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 12: Куча выдает H-I (вес 12)',
 description: 'Плесень тянется к последней чистой
 type: 'info'
 }, ...prev]));
},

```

```

 },
 // Step 2: Concurrently merge into Kolkhozes (Component 1) => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/blue)
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' };
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' };
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы',
 description: 'Все выбранные дороги построены радостно',
 type: 'success'
 }, ...prev]);
 },
 // Step 3: Five-Year Plan 2 (Inspecting bridge between components) => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}:
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 status: 'included', color: 'red'
 } : e));
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мост',
 description: 'Теперь Красный и Синий колхозы одружились',
 type: 'warning'
 }, ...prev]);
 },

```



```

 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 if (algo === 'kruskal') {
 setLogs([
 {
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа
 изначально имеют свой собственный цвет.
 Мы хотим раскрасить граф так, чтобы
 никакие две смежные вершины не имели
 одинакового цвета.',
 type: 'info',
 },
]);
 } else if (algo === 'prim') {
 setLogs([
 {
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стар
 тельную вершину (Heap) и захватываем соседей!',
 type: 'info',
 },
]);
 } else {
 setLogs([
 {
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 д
 ружек имеет своего старшего. Каждая д
 ружка хочет объединиться с соседями,
 чтобы объединиться в колхоз.',
 type: 'info',
 },
]);
 }
};

```

```

const getColorClass = (color: string, id: number) => {
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 tex
 case 'blue': return 'bg-blue-500 border-blue-300
 }
}

```

```

 Наблюдай за работой Краскала (Раскраска),
 </p>
</div>

```

```

{/* Algorithm Tabs */}
<div className="flex flex-wrap items-center gap-2">
 <div className="flex items-center bg-slate-400 p-2 rounded">
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +

```

```
</div>
```

```
<div className="grid grid-cols-1 lg:grid-cols-3
 /* Graph Canvas */
 <div className="lg:col-span-2 bg-slate-950 rounded-3xl
 active min-h-[480px] overflow-hidden">

 /* Legend */
 <div className="absolute top-4 left-4 bg-slate-900
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
 {activeAlgo === 'kruskal' ? (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
 </div>
 </div>
 </div>
) : activeAlgo === 'prim' ? (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-1 bg-sky-600">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
 </div>
 </div>
) : (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-400">
 </div>
```

```

 />
 <text
 x={{u.x + v.x} / 2 + "%"}
 y={{u.y + v.y} / 2 + "%"}
 fill={isInspecting ? '#f59e0b' :
: '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
 }}}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
 {vertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: v
 className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'p
 <span className="text-[8px] block -
 }}
 </div>
)}}
</div>

```

```

 }}
 </div>
</div>
)}}

<div className="flex-1 p-4 overflow-y-auto"
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-a
 {log.type === 'success'
 ? 'bg-emerald-950/40 border-emera
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-5
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500
 : 'bg-slate-900/60 border-slate-7
 }
 }
 >
 <div className="flex items-center gap
 {log.type === 'success' && <CheckCi
 {log.type === 'warning' && <HelpCir
 {log.type === 'error' && <XCirc
 {log.type === 'info' && <Play class
 <h5 className="font-bold text-sm le
 </div>
 <p className="text-xs text-slate-300
 </div>
)}}
</div>
</div>
</div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
 <div className="flex flex-col sm:flex-row items

```

```

 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-indigo-400">
 </p>
 <p className="text-2xl font-black text-white">
 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 ожении:</p>
 <p className="text-xs text-slate-300 leading-5">
 ди и красим вершины в один цвет, бракуя циклы.
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-indigo-400">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
 if parent[i] == i: return i
 parent[i] = find(parent[i])
 return parent[i]

def union(i, j):
 r_i, r_j = find(i), find(j)
 if r_i != r_j:
 parent[r_i] = r_j
 return True
 return False

2. Жадный выбор ребер
edges.sort() # O(E log E)

```

```

</p>
<pre className="text-xs text-emerald-400" style="font-family: monospace; font-size: 0.8em; margin: 0; padding: 0 10px;">
80 flex-1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}

</pre>
</div>
</div>
</div>
))

{activeCheatTab === 'boruvka' && (
 <div className="grid grid-cols-1 lg:grid-cols-2" style="background-color: #f0f0f0; padding: 10px; border-radius: 10px;">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl" style="background-color: #1e293b; color: white;">
 <p className="text-slate-400 text-xs font-mono">
 <Clock className="w-3.5 h-3.5 text-rose-500" />
 </p>
 <p className="text-2xl font-black text-white">
 Boruvka
 </p>
 <p className="text-slate-400 text-xs font-mono">
 m
 </p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl" style="background-color: #1e293b; color: white;">
 <p>

```

```
 </div>
)}
 </div>

 </div>
);
};
```

---

ФАЙЛ 51/81: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const cards = [
 {
 img: dijkstraImg,
 alt: 'Добрый робот Дейкстра',
 title: ' Добрый (Дейкстра)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'положительными',
 highlightColor: 'text-emerald-400',
 rest: 'весами. Подсунь отрицательное ребро – сломает',
 complexity: 'O((V+E) log V)',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый – зато тащит',
 highlight: 'отрицательные',
```



```
 </div>
)})
 </div>
);
}
```

---

ФАЙЛ 52/81: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы — показываем в кнопке, что */
 currentTitle: string;
 index: number;
 total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
 const [open, setOpen] = useState(false);

 useEffect(() => {
 document.body.style.overflow = open ? "hidden" : "";
 return () => {
 document.body.style.overflow = "";
 };
 }, [open]);
```

```
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
</div>
)}
</>
);
};
```

---

ФАЙЛ 53/81: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Code } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
 activeTab: 'guide' | 'simulator';
 setActiveTab: (tab: 'guide' | 'simulator') => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab, setActiveTab }) => {
 const [busy, setBusy] = useState(false);
```

```
 const saveBook = async () => {
```

```

>
 <BookOpen className="w-4 h-4" /> Учебное по
</button>
<button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg s
 : 'text-slate-400 hover:text-white'
 }`}
>
 <Cpu className="w-4 h-4" /> Тренажёры
</button>
<a
 id="download-pdf-btn"
 href="/export/full_code_all_pages.pdf"
 download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
over:bg-emerald-600/20 border border-emeral
 title="Скачать полный PDF сборник всех стра
>
 <FileDown className="w-4 h-4" /> Скачать PD

<a
 id="download-txt-btn"
 href="/export/full_code_all_pages.txt"
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодо
>
 <FileText className="w-4 h-4" /> TXT

```

```

 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.f
 function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 function ccw(ax:number,ay:number,bx:num
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1]
 if (!crosses.includes(i)) crosses.p
 if (!crosses.includes(j)) crosses.p
 }
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(
 const xing = crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x
 stroke={xing ? "#f43f5e" : "#4f46e5"}
 }
 }
 const labels = ["A","B","C","D","E"];

```

```

 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=0;j<edges.length;j++)
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 }
 const labels = ["\u04141", "\u04142", "\u04143"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#f43f5e" stroke-width={2}/>
 <text x={p.x} y={p.y+3} textAnchor="middle">{p.id}</text>
 </g>);
 }
 return <>{lines}{circles}</>;
 })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20" style={z-20">
 V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

```

```

<div className="bg-amber-950/40 border border-amber-950/40 padding-10">
 ▲ Запомни: K5 и K3,3 – два кита непланарности.
 гомеоморфный K5 или K3,3 – он непланарен. Теорема Кёрши.
</div>

```

```

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '🍌', color: 'red',
 голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: '🍲', color: 'blue',
 ишу ИИ за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'green',
 ез сметаны, плиз.', animState: 'idle' },
]));

 const [servedHistory, setServedHistory] = useState<string[]>([]);
 const [isServing, setIsServing] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string[]>([]);

 const addLog = (msg: string) => setLog(prev => [...prev, msg]);

 // ENQUEUE: Встать в конец очереди (хвост / Tail)
 const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяжёло!');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
 }, [isServing, queue.length]);

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

 const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
 };

 setQueue(prev => [...prev, newGuest]);

```

```

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: '🍲', color: 'indigo', text: 'У меня нет голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: '🍲', color: 'teal', text: 'Я напишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: '🐈', color: 'red', text: 'Дайте котика без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!']);
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState: 'out' })), 500);
 });
};

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-indigo-500 p-6">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 В очереди за супом всё честно. Кто первый встал, тот и ест. Новые гости встают строго в конец очереди. Здесь нет обхода и блата – только строгий порядок.
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={enqueue}

```

```

<div className="flex items-end justify-between">
```

```
 { /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
 <div className="flex flex-col items-center">
 <div className="relative">
 <div className="absolute -top-10 left-10">
 <div className="w-1.5 h-6 bg-slate-400">
 <div className="w-2 h-4 bg-slate-400">
 </div>

 </div>
 <div className="text-center">
 <div>

 ПОВАР

 </div>
 </div>
 </div>
```

```
 { /* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */ }
 <div className="flex flex-col items-center">

 </div>
```

```
 { /* СПИСОК ОЧЕРЕДИ */ }
 <div className="flex-1 flex items-end gap-2">
 {queue.length === 0 ? (
 <div className="flex-1 flex flex-col justify-between">
 <div>
 <p className="text-xs font-bold font-mono tracking-wider">
 <p className="text-[10px]">Все сыты
 </div>
) : (
 queue.map((customer, idx) => {
 const isHead = idx === 0;
 const isTail = idx === queue.length - 1;
```



```


 })
 </div>
 </div>
);
})
})
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */}
<div className="md:col-span-3 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[10px] font-mono"
 Сытые гости (Архив FIFO)
 </div>

 <div className="absolute top-4 right-4 flex items-center justify-center gap-1
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse">
 СЫТЫ
 </div>

 <div className="flex-1 flex flex-col justify-between p-4"
 {servedHistory.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center gap-2"

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-1.5 w-full"
 {servedHistory.map((item, idx) => {

```

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';

interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR', 'FISH', 'SALMON']);
 const [newWord, setNewWord] = useState('');
 const [nodes, setNodes] = useState<{ [key: number]: TrieNode }>({});
 const [simulationStep, setSimulationStep] = useState<number>(0);
 const [bfsQueue, setBfsQueue] = useState<number[]>([]);
 const [currentNode, setCurrentNode] = useState<number>(0);
 const [speechText, setSpeechText] = useState<string>('Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и начни симуляцию!');
 const [isTalking, setIsTalking] = useState<boolean>(false);
 const [isBlinking, setIsBlinking] = useState<boolean>(false);

 // Регулярное моргание рыбы
 useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 1000);
 return () => clearInterval(blinkInterval);
 }, []);
};
```

```

 }
 if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
 }
 });

 let currentX = 0;
 const paddingX = 70;
 const paddingY = 80;

 const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : (leftX + rightX) / 2;
 if (leftX === -1) currentX++;
 }
 };

 layoutDFS(0, 0);

 setNodes(newNodes);
 setSimulationStep(0);
 setBfsQueue([]);
 setCurrentNode(null);
 setSpeechText(

```

```

 rootChildren.forEach(({childId}) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на графике

 уже суффикса просто нет. Жми «Следующий шаг»!'
);
 };

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полный

 граф. Теперь можно полюбоваться таблицей переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.children.length} детей)`;

 for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] != null) {
 v = updatedNodes[v].fail;
 }
 updatedNodes[rNode.fail] = { children: { ...rNode.children, [char]: u }, fail: v };
 logs += `Добавляем ребро ${r} -> ${u} с суффиксом ${char}`;
 }

 setBfsQueue(newQueue);
 setNodes(updatedNodes);
 setSimulationStep(3);
 setCurrentNode(rNode);
 setSpeechText(logs);
};

```

```

 </div>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate
 transition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center just
 <div className="w-40 h-40 relative flex items
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_
]`}
 >
 {/* Тело лосося */}
 <ellipse cx="100" cy="100" rx="70" ry="45"
 <path d="M 40 85 Q 100 55 160 85 Q 100 115
 40 85"
 fill="#be123c"
 className="origin-[35px_100px] animate-
 />

```

```


 </div>

 {/* Пузырь с текстом (Баббл) */}
 <div className="md:col-span-2 bg-slate-800 border-2 border-t-transparent min-h-[140px]">
 {/* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -transform rotate(-45deg) border-b-[12px] border-b-transparent border-l-[12px] border-l-transparent border-r-[12px] border-r-transparent border-t-[12px] border-t-transparent" />

 <div className="flex items-center space-x-2 text-white">
 <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`} />
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leading-relaxed">
 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-slate-700">
 <div className="text-xs text-slate-400 flex justify-between">
 <Info className="w-3.5 h-3.5 text-blue-400" />
 Статус: {simulationStep === 0 ? 'Бор готов к старту!' : 'Бор занят!'}
 </div>

 <div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 text-white px-3 py-1 shadow-lg shadow-emerald-900/40 transition" />
 <Play className="w-4 h-4" /> Запустить
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stopBFS}
 className="bg-rose-600 hover:bg-rose-500 text-white px-3 py-1 shadow-lg shadow-rose-900/40 transition" />
 <Stop className="w-4 h-4" /> Остановить
 </button>
)}
 </div>
 </div>
 </div>

```

```

return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
 <defs>
 <marker id="arrowhead" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 <marker id="fail-arrow" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 </defs>

 {/* Draw Edges */}
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x ||

 const isCurrent = currentNode === node.

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={({parent.x + node.x} / 2 - 10)}
 y={({parent.y + node.y} / 2}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}

```

```

let fill = '#1e293b';
let stroke = '#475569';

if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
} else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
} else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
}

return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && (
 <text
 x={node.x + 12}

```



```

 }}}
 </div>
</div>

<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
</form>
</div>

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex items-center" >

 Таблица переходов и fail-

 Всего вершин: {Object.keys(nodes).length}

 </h5>

 <div className="max-h-[220px] overflow-y-auto" >
 <table className="w-full text-left border-collapse" >
 <thead>

```

```

 #{node.id}
 {node.id === 0 && <span className=
</td>
<td className="py-2.5 px-3">
 <span className="bg-slate-800 b
 {node.char}

</td>
<td className="py-2.5 px-3 text-s
 {node.id === 0 ? '-' : `#${node
</td>
<td className="py-2.5 px-3">
 {node.id === 0 ? {
 <span className="text-slate-5
 } : {
 <span className="bg-indigo-95
 #{node.fail}

 }}
</td>
<td className="py-2.5 px-3">
 {node.id === 0 || node.term_lin
 <span className="text-slate-5
 } : {
 <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Ск
 #{node.term_link}

 }}
</td>
<td className="py-2.5 px-3 text-s
 {Object.keys(node.children).len
 ? 'ø'
 : Object.entries(node.children
 .map(([c, id]) => `${c}→#
 .join(', '))}
</td>
<td className="py-2.5 px-3">

```

```

 result?: number; // partial
}
```

```
// ===== PURE LOGIC: build tree =====
```

```
function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}
```

```
// ===== STEP GENERATORS =====
```

```
// 1. Build steps (recursive top-down)
```

```
function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};

 const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5
 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];
};
```

```

 codeLine: 2, treeState: tree, highlightNodes: [v]
 }
 return 0;
}
if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}] = ${tree[v]}`, codeLine: 4, treeState: tree, highlightNodes: [v]
 return tree[v];
}
const m = (l + r) >> 1;
steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`, codeLine: 6, treeState: tree, highlightNodes: [v]
const left = go(2 * v, l, m, ql, qr);
const right = go(2 * v + 1, m + 1, r, ql, qr);
const res = left + right;
steps.push({ id: id++, desc: `Возврат в узел ${v}:`, codeLine: 8, treeState: tree, highlightNodes: [v],
 rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [2 * v, 2 * v + 1]
return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]`, codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans
return steps;
}

```

```

// 3. Bottom-Up Query steps (iterative on implicit tree)
function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

 const steps: Step[] = [];
 let id = 0;
 let l = qL + n;
 let r = qR + n + 1; // half-open [l, r)
 let res = 0;

```

```

 v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l == r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
];

```

```

const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",
 " // -----",
 " if (ql <= l && r <= qr) return tree[v];",
 " // -----",
 " mid = (l + r) >> 1;",
 " // спускаемся в обоих детей",
 " return query(left) + query(right);",
 "}"
];

```

```

const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " if (r & 1) res += tree[--r];",
 " l >>= 1; r >>= 1;",
 " }",
 " return res; }"
];

```

```

];

```



```
</pre>
```

```

{ /* Description + result */
<div className="px-3 py-2.5 bg-slate-900/60 border-t-1 border-l-1 border-r-1 border-b-1 border-slate-900/60" style={{
 px: 3, py: 2.5, bg: 'slate-900/60', border: '1px solid slate-900/60' }}>
 <span className={`mt-0.5 inline-block w-2 h-2 rounded-sm`} style={{
 mt: 0.5, display: 'inline-block', width: 2, height: 2, border: '1px solid slate-900/60',
 background: 'slate-900/60' }}>
 {step.desc}
 {step.result !== undefined && {
 <span className={`ml-auto shrink-0 font-mono text-sm`} style={{
 ml: 'auto', shrink: 0, font: 'monospace', font: '12px', color: 'emerald' ? 'text-emerald-300' : 'text-slate-400' }}>
 {step.result}

 }}
</div>

```

```

{ /* Controls */
<div className="px-3 py-2.5 bg-slate-950 border-t-1 border-l-1 border-r-1 border-b-1 border-slate-950" style={{
 px: 3, py: 2.5, bg: 'slate-950', border: '1px solid slate-950' }}>
 <div className="flex items-center bg-slate-900/60 border-t-1 border-l-1 border-r-1 border-b-1 border-slate-900/60" style={{
 px: 3, py: 2.5, bg: 'slate-900/60', border: '1px solid slate-900/60' }}>
 <button onClick={() => { setPlaying(false); setSpeed(1200); }} style={{
 px: 1.5, py: 1, rounded: 0.5, text: 'slate-400', hover: 'text-white', hover: 'bg-slate-800', rounded: 0.5 }}>
 <RotateCcw className="w-3 h-3" />
 </button>
 <button onClick={() => { setPlaying(false); setSpeed(700); }} style={{
 px: 1.5, py: 1, rounded: 0.5, text: 'slate-400', hover: 'text-white', hover: 'bg-slate-800', rounded: 0.5 }}>
 <RotateCw className="w-3 h-3" />
 </button>
 <button onClick={() => setPlaying(!playing)} style={{
 px: 1.5, py: 1, rounded: 0.5, text: 'white', transition: 'all 0.2s', background: playing ? 'bg-rose-500' : 'bg-slate-800', rounded: 0.5 }}>
 {playing ? <Pause className="w-3 h-3" /> : <Play className="w-3 h-3" />}
 </button>
 </div>
 <div style="margin-top: 10px;">
 <select value={speed} onChange={e => setSpeed(Number(e.target.value))} style={{
 text: '[9px] font-bold rounded px-1.5 py-1 outline-none',
 width: 100, height: 30 }}>
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
 </div>
</div>

```

```

const buildSteps = useMemo(() => genBuildSteps(arr),
const queryTDSteps = useMemo(() => genQueryTopDownSteps(arr),
const queryBUSteps = useMemo(() => genQueryBottomUpSteps(arr),

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
 <div className="bg-slate-900 rounded-2xl border border-indigo-500" >
 /* ---- TOP BAR: array editor + query range ---- */
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4">
 /* Array input */
 <form onSubmit={handleApply} className="flex flex-col">
 <div className="flex items-center justify-between">
 <label className="text-[10px] font-bold uppercase">
 Array input
 </label>
 <button type="button" onClick={randomize} className="text-white">
 Refresh
 </button>
 </div>
 <div className="flex gap-2">
 <input
 type="text"
 value={customInput}
 onChange={e => setCustomInput(e.target.value)}
 className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo-600 text-white px-4 py-2">
 Применить
 </button>
 </div>
 <div className="flex flex-wrap gap-1.5 pt-1">
 {arr.map((v, i) => (

 A[{i}] = {v}

))}

 Total sum: {totalSum}

 </div>
 </form>
 </div>
 </div>
)
)

```



```

{ /* ---- TABS ---- */
<div className="flex flex-wrap items-center gap-1"
 <button onClick={() => setView('build')} className="border-indigo-500 border-2 p-2"
 -colors ${view === 'build' ? 'border-indigo-500' : 'border-slate-200'}`>
 <Hammer className="w-3.5 h-3.5" /> Построить
 </button>
 <button onClick={() => setView('queries')} className="border-emerald-500 border-2 p-2"
 on-colors ${view === 'queries' ? 'border-emerald-500' : 'border-slate-200'}`>
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('theory')} className="border-blue-500 border-2 p-2"
 n-colors ${view === 'theory' ? 'border-blue-500' : 'border-slate-200'}`>
 <Info className="w-3.5 h-3.5" /> Почему / Сравн
 </button>
</div>

```

```

{ /* ---- TAB CONTENT ---- */
{view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=""
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
)}

```

```

{view === 'queries' && (
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху

```

```

 </div>
 </div>

 { /* Comparison cards */ }
 <div className="grid grid-cols-1 xl:grid-cols-2" >
 <div className="bg-slate-950 p-5 rounded-xl" >
 <div className="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md" >
 ↑ Запрос Сверху-вниз (Рекурсия)
 </div>
 <p className="text-xs text-slate-300 mb-3" >
 Начинаем с корня. Если отрезок узла полон, то идем в левую ветку. Иначе спускаемся в обоих детей.
 </p>
 <div className="space-y-1.5 text-xs" >
 <div className="flex justify-between border-b border-slate-700" >
 0(
 <div className="flex justify-between border-b border-slate-700" >

 <div className="flex justify-between" >
 ✓ легко</div>
 </div>
 </div>
 </div>
 </div>
 <div className="bg-slate-950 p-5 rounded-xl" >
 <div className="absolute -top-3 left-4 bg-amber-500 border border-amber-500 shadow-md" >
 ↑ Запрос Снизу-вверх (Итерация)
 </div>
 <p className="text-xs text-slate-300 mb-3" >
 Стартуем с двух указателей (l и r) на левом и правом узлах. Идем по левому узлу до его левого соседа. Поднимаемся пока l {".
 </p>
 <div className="space-y-1.5 text-xs" >
 <div className="flex justify-between border-b border-slate-700" >

 <div className="flex justify-between border-b border-slate-700" >

 <div className="flex justify-between" >
 ✓ легко</div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

```

```
export const Sidebar: React.FC<SidebarProps> = ({ selectQuery }) => {
 const [query, setQuery] = useState("");
```

```
 const groups = useMemo(() => {
 const q = query.trim().toLowerCase();
 const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
 const map = new Map<string, typeof chapters>();
 for (const c of filtered) {
 const key = c.category?.trim() || "Прочие темы";
 if (!map.has(key)) map.set(key, []);
 (map.get(key) as typeof chapters).push(c);
 }
 return Array.from(map.entries());
 }, [query]);
```

```
 return (
 <aside className="w-full bg-slate-900/80 lg:bg-transparent lg:p-4">
 <div className="p-4 pb-3 shrink-0">
 <h3 className="text-xs font-bold text-slate-400">
 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 <div className="relative">
 <Search className="w-4 h-4 text-slate-500 absolute right-0 top-0">
 <input
 type="text"
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам..."
 className="w-full bg-slate-800/70 border border-slate-500 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 type="button"
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -translate-y-1/2 text-slate-500"
 aria-label="Очистить поиск"
 />
)}
 </div>
 </div>
 </div>
 </aside>
);
}
```

```

 }` }
 />
 </button>
);
 }}}
</div>
</div>
)})
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-l
] text-slate-300 space-y-1.5 hidden lg:block">
 <div className="font-bold text-indigo-400 flex .
 <Sparkles className="w-3.5 h-3.5" /> Как поль
 </div>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по
 симулятора.
 </p>
 <p className="text-slate-400 italic">Стрелки ←
 </div>
</aside>
);
};

```

---

ФАЙЛ 59/81: `src/components/Simulator.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

```

```

interface Plate {
 id: string;
 name: string;
 icon: string;

```

```

 });
 const [newCustomText, setNewCustomText] = useState('');
 const [newCustomProb, setNewCustomProb] = useState(0);
 const [heapMessage, setHeapMessage] = useState<string>('');

 // Stack handlers
 const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: '🥞' },
 { name: 'Кастрюля от супа', icon: '🍲' },
 { name: 'Чашка от кофе', icon: '☕' },
 { name: 'Миска с остатками салата', icon: '🥗' },
 { name: 'Тарелка от тако', icon: '🌮' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(`Положили поверх стопки: ${newPlate.name}`);
 };

 const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name}`);
 };

 const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
]);
 };

```

```

 });
 setDequeueMessage(" Очередь восстановлена.");
 };

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью
 `);

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(` Выбран самый "тяжелый" кандидат:
 `);

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный

```

```


 DFS

 </button>

 <button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between ${activeTab === 'queue' ? 'bg-indigo-600/20 border-indigo-500 text-white' : 'bg-slate-800/60 border-slate-700/60 text-white'} `}
 >
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-white' : 'text-slate-400'} `} />
 <div className="text-left">
 <div className="font-bold text-sm text-white">Queue</div>
 <div className="text-xs opacity-80">FIFO</div>
 </div>
 </div>

 BFS

 </button>

 <button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between ${activeTab === 'heap' ? 'bg-emerald-600/20 border-emerald-500 text-white' : 'bg-slate-800/60 border-slate-700/60 text-white'} `}
 >
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'text-white' : 'text-slate-400'} `} />
 <div className="text-left">
 <div className="font-bold text-sm text-white">Heap</div>
 <div className="text-xs opacity-80">Приоритет</div>
 </div>
 </div>
 </button>
 </div>
 </div>
</div>

```

```

 onClick={resetStack}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>

 {popMessage && {
 <div className="bg-blue-950/60 border borde
imate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {popMessage}
 </div>
 }}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
 >
 {stack.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стон
 <p className="text-xs mt-1">Добавьте гр
 </div>
 } : {
 <div className="w-full max-w-sm flex flex
 <div className="absolute top-0 text-xs
full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 anim
 </div>

 {/* Reversed map so top of stack is vis
 {stack.slice().reverse().map((plate, ind
 const isTop = index === 0;

```



```

<div>
 <h3 className="text-lg font-bold text-white">
 Симуляция очереди за супом

 /span>
 </h3>
 <p className="text-slate-400 text-xs mt-1">
 Кто первым пришел, тот первым получил суп
 </p>
 </div>
 <div className="flex flex-wrap items-center">
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex item-center justify-center gap-2.5 rounded-xl text-sm font-bold shadow-lg"
 >
 <Plus className="w-4 h-4" /> Встать в очередь
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex item-center justify-center gap-2.5 rounded-xl border-indigo-500/40 px-4 py-2.5 rounded"
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-slate-700 text-white transition-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
</div>

```

```

{dequeueMessage && {
 <div className="bg-indigo-950/60 border border-indigo-950/60 padding-2">

```

```

 >
 {isFirst && {
 <span className="absolute -top-
se tracking-wider shadow">
 Первый (Head)

 }}
 {
 <span className={`font-bold text-
 {person.name}

 <span className={`text-[10px] fon
 isFirst ? 'bg-indigo-500/30 tex
 }`}>
 {isFirst ? 'Получает сун' : `В

 </div>
 });
 }}}

 <div className="flex flex-col items-cen
-600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 <span className="text-xs font-mono up
 </div>
 </div>
 })
 <div className="mt-6 text-xs text-slate-500
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
 </div>
 </div>
 </div>
 })

 {/* HEAP TAB */}
 {activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x

```

```

 onChange={e => setNewCustomText(e.target.value)}
 placeholder='например: "Привет, я лучший"'
 className="w-full bg-slate-900 border border-emerald-500 transition-colors"
 />
 </div>
 <div className="md:col-span-3">
 <label className="block text-xs font-bold">
 Вероятность:
 </label>
 <input
 type="range"
 min="0.01"
 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e.target.value))}
 className="w-full accent-emerald-500 cursor-pointer"
 />
 </div>
 <div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center gap-2 bg-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 <Plus className="w-4 h-4" /> Добавить в список
 </button>
 </div>
 </form>

```

```

{heapMessage && (
 <div className="bg-emerald-950/60 border border-emerald-500/40 gap-3 animate-fade-in">

 </div>
)}

```



```

const q = query.trim().toLowerCase();
if (!q) return all;
return all.filter(
 (i) =>
 i.entry.title.toLowerCase().includes(q) ||
 (i.entry.hint ?? "").toLowerCase().includes(q)
 (i.chapterTitle ?? "").toLowerCase().includes(q)
);
}, [query]));

return (
 <div>
 <div className="mb-6">
 <h2 className="text-xl sm:text-2xl font-extrabold">
 <p className="text-sm text-slate-400 leading-relaxed">
 Все интерактивные демонстрации в одном месте.
 здесь их можно открыть напрямую.
 </p>
 </div>

 <div className="relative mb-6 max-w-md">
 <Search className="w-4 h-4 text-slate-500 absolute">
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск тренажёра..."
 className="w-full bg-slate-900 border border-
 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -translate-y-1/2"
 aria-label="Очистить поиск"
 >
 <X className="w-4 h-4" />
 </button>
)}
 </div>
 </div>
 </div>
)

```

```
 });
};
```

---

ФАЙЛ 61/81: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";
```

```
/**
```

```
 * Песочница поворотов.
```

```
 *
```

```
 * Здесь ничего не подсказывается: дано настоящее дерево.
```

```
 * поднять отмеченный узел в корень. Клик по узлу = один шаг.
```

```
 * Порядок поворотов выбираешь сам; правильный ли он пока неясно.
```

```
 * только после того, как узел окажется наверху (или по крайней мере в середине).
```

```
 */
```

```
export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}
```

```
const leaf = (key: number): TNode => ({ key, left: null, right: null });
```

```
const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null;
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и просека (zig-zag-zig-zag)
```

```
export const PRESETS: { name: string; hintCase: string; target: number; }[] =
```

```
{
```

```
 name: "Линия влево",
```

```
 hintCase: "Zig-Zig",
```

```
 target: 1,
```

```
export const findPath = (root: TNode | null, key: number): TNode[] => {
 const path: TNode[] = [];
 let cur = root;
 while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};
```

```
/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
}
```

```
/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
 const path = findPath(root, key);
```

```

const rows = Math.max(1, Math.max(...nodes.map((n) =>
const colW = 46;
const rowH = 62;
return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + co
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
});
}

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.l
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; co
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

 const { nodes, edges, width, height } = useMemo(() =>
 const path = useMemo(() => new Set(findPath(tree, tar

```



```

 >
 <Dices className="w-3.5 h-3.5" /> Другое дере
 </button>
</div>

```

```

<div className="flex gap-2 mb-3 overflow-x-auto n
 {PRESETS.map((p, i) => {
 <button
 key={p.name}
 onClick={() => reset(i)}
 className={`px-3 py-1.5 rounded-lg text-[11p
 i === presetIdx ? "bg-indigo-600 text-whi
 }`}
 >
 {p.name}
 </button>
)})}
</div>

```

```

<div className="bg-slate-900 rounded-xl border bo
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420)
 role="img"
 >
 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}

```

```

<div className="flex flex-wrap items-center gap-2"
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-1
 -white disabled:opacity-40 text-xs font-bol
 >
 <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
 </button>
 <button
 onClick={() => reset()}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-1
 showAnswer
 ? "bg-amber-600/20 border-amber-500 text-
 : "bg-slate-800 border-slate-700 text-sla
 }`}
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5"
 {showAnswer ? "Скрыть разбор" : "Сдаться / по
 </button>

 <span className="text-[11px] text-slate-500 ml-
 поворотов: {moves.length} • минимум: {minimal

</div>

```

```

{showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40
 Сейчас splay крутил бы узел{" "}
 {expected
 </div>

```

---

ФАЙЛ 62/81: src/components/SplayWalkViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect, useMemo, useRef, useState } from 'react';
```

```
/**
```

```
 * Splay на БОЛЬШОМ дереве – пошагово.
```

```
 *
```

```
 * Проблема, которую решает: на игрушечных A/B/C-схемах
```

```
 * «двух зигов подряд», а мгновенный splay в большом дереве
```

```
 * стало». Здесь каждый ОДИНОЧНЫЙ поворот – атомарный шаг.
```

```
 * поворот → текст, какие рёбра развернулись и какие поворачиваются.
```

```
 * Порядок поворотов виден явно: Zig-Zig – ВЕРХНЕЕ ребро, Zig-Zag –
```

```
 * НИЖНЕЕ первым.
```

```
 *
```

```
 * Раскладка – слоты in-order: ключи не двигаются по горизонтали.
```

```
 * поворот меняет только глубину. Поэтому «что переехало» не важно.
```

```
 *
```

```
 * Чистые функции (splaySteps, rotateUp, collectEdges) – не зависят от
```

```
 */
```

```
export interface SN {
```

```
 key: number;
```

```
 left: SN | null;
```

```
 right: SN | null;
```

```
 parent: SN | null;
```

```
}
```

```
export function cloneWithParents(t: SN | null): SN | null {
```

```
 if (!t) return null;
```

```
 const n: SN = { key: t.key, left: null, right: null, parent: null };
```

```
 const l = t.left ? cloneWithParents(t.left) : null;
```

```
 const r = t.right ? cloneWithParents(t.right) : null;
```

```
 if (l) {
```

```
 n.left = l;
```

```

 child.right = p;
 p.parent = child;
 } else {
 p.right = child.left;
 if (child.left) child.left.parent = p;
 child.left = p;
 p.parent = child;
 }
 child.parent = g;
 if (!g) return;
 if (g.left === p) g.left = child;
 else g.right = child;
}

/** Рёбра в направленном виде "p→c": для diff между шагами
export function collectEdges(t: SN | null): string[] {
 const es: string[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
 };
 walk(t);
 return es.sort();
}

const diffEdges = (before: string[], after: string[]): string[] {
 const rm = before.filter((e) => !after.includes(e));
 const ad = after.filter((e) => !before.includes(e));
 const parts: string[] = [];
 for (const a of ad) {
 const [p, c] = a.split("→");
 if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}→${c} удалено`);
 else parts.push(`поддерево ${c} переехало к ${p}`);
 }
 return parts.join("; ");
}

```

```

 cur = key < cur.key ? cur.left : cur.right;
 }
 const x = found ? cur : last;
 steps.push({
 kind: "search",
 root: cloneWithParents(root)!,
 pathKeys: [...pathKeys],
 note: isInsert
 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск упёрся в ${last.key}
 rotCount: 0,
));

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
 while (root && root.parent) root = root.parent;
};
/** Один поворот = два атомарных кадра: cut (что пере
const emitRotation = (child: SN, label: string) => {
 const p = child.parent!;
 const mid = p.left === child ? child.right : child.left;
 steps.push({
 kind: "cut",
 root: cloneWithParents(root)!,
 cutEdge: [child.key, mid ? mid.key : child.key],
 cutLabel: mid ? `поддерево ${mid.key}` : "пустое
 caseLabel: label,
 note: mid
 ? `Отстёгиваем среднее поддерево ${mid.key}: он
 сторону к ${p.key}.`
 : `Среднего поддерева нет: ${child.key} поднима
 rotCount,
));
 const before = collectEdges(root);

```

```

 caseLabel: "Zig-Zig",
 note: `Zig-Zig (${side}, линия ${g.key}—${p.key}
 а ${target.key} пока не трогаем.`,
 rotCount,
 });
 emitRotation(p, "Zig-Zig");
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zig",
 note: `Теперь обычный зиг: ребро ${target.parent.key}—${target.key}
 не трогаем.`,
 rotCount,
 });
 emitRotation(target, "Zig-Zig");
} else {
 const side = `${g.key} → ${pIsLeft ? "влево" : "вправо"}
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [p.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ
 к предыдущему.`,
 rotCount,
 });
 emitRotation(target, "Zig-Zag");
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Второе ребро: ${target.parent.key}—${target.key}
 не трогаем.`,
 rotCount,
 });
 emitRotation(target, "Zig-Zag");
}
}
}

```

```

 { line: 6, text: " rotate(p over g) # BE
 { line: 7, text: " rotate(x over p)" },
 { line: 8, text: " else: # Zi
 { line: 9, text: " rotate(x over p) # HI
 { line: 10, text: " rotate(x over g)" },
];

/* ----- рендер -----

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

const layout = (root: SN | null) => {
 const pos = new Map<number, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: SN | null, d: number) => {
 if (!n) return;
 walk(n.left, d + 1);
 pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d });
 slot += 1;
 walk(n.right, d + 1);
 };
 walk(root, 0);
 return pos;
};

/** Врезка «механика одного поворота»: как вправляют вы
const MechanismInset: React.FC<{ step: SplayStep | null }> = ({ step }) => {
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";
 return (
 <div className="bg-slate-950/80 rounded-lg border border-slate-700">
 <h4 className="text-xs font-bold text-slate-300 mb-2">Механика поворота
 <p className="text-[10px] text-slate-500 mb-2">Каждый поворот состоит из
 <svg viewBox="0 0 340 190" className="w-full">
 <g transform="translate(10,6)">
 <text y="10" fill="#94a3b8" fontSize="11" fontStyle="italic">

```

```

 }</text>
 <text x="28" y="160" fill="#64748b" fontSize=
 </g>
</svg>
<p className="text-[10px] text-slate-500 leading-
 Порядок всегда один: отстегнуть β у x → поднять
</p>
</div>
);
};

```

```

const SplayWalkViz: React.FC = () => {
 const initial = useMemo(buildInitial, []);
 const [steps, setSteps] = useState<SplayStep[] | null>();
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [input, setInput] = useState("");
 const [treeVers, setTreeVers] = useState(0);
 const baseRef = useRef<SN>(initial);
 const timer = useRef<ReturnType<typeof setTimeout> | null>();

```

```

 const runKey = (key: number, isInsert = false) => {
 const { steps: st } = splaySteps(baseRef.current, key);
 setSteps(st);
 setIdx(0);
 setPlaying(true);
 };

```

```

 useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 // фиксируем результат в базовом дереве
 baseRef.current = steps[steps.length - 1].root ?? null;
 setTreeVers((v) => v + 1);
 return;
 }
 timer.current = setTimeout(() => setIdx((v) => v + 1), 100);
 }, [playing, steps, idx, timer]);

```



```

const hBefore = useMemo(() => (steps ? heightOf(steps) : 0));
const hNow = heightOf(viewRoot);

const doInsert = () => {
 const v = parseInt(input, 10);
 if (Number.isNaN(v)) return;
 const withNew = bstInsert(baseRef.current, v);
 baseRef.current = withNew;
 setTreeVers((t) => t + 1);
 runKey(v, true);
 setInput("");
};

const stepColor = (k: number): string => {
 if (!step) return "#0f172a";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#f59e0b";
 if (step.aimEdge && step.aimEdge.includes(k)) return "#f59e0b";
 return "#0f172a";
};

const strokeColor = (k: number): string => {
 if (!step) return "#64748b";
 if (step.kind !== "search" && step.pathKeys?.includes(k)) return "#f59e0b";
 if (step.aimEdge?.includes(k)) return "#f59e0b";
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 {/* Wanka */}
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Кликните по элементу, чтобы добавить его в дерево
 </h3>
 </div>
 </div>
 </div>
 </div>
);

```

```

 className="px-3 py-2 rounded-lg text-xs font-medium"
 >
 Перемешать
 </button>
 </div>
 </div>

 { /* Плеер */ }
 { steps && (
 <div className="flex flex-wrap items-center gap-2" >
 <button onClick={() => { setIdx(0); setPlaying(true); }}
 className="px-3 py-1.5 rounded-lg text-xs font-medium bg-slate-700 text-white disabled:opacity-50"
 > В начало
 </button>
 <button onClick={() => { setPlaying(false); setPlaying(true); }}
 className="px-3 py-1.5 rounded-lg text-xs font-medium bg-slate-700 text-white disabled:opacity-50"
 > Пауза
 </button>
 <button onClick={() => setPlaying(!playing)}
 className="px-3 py-1.5 rounded-lg text-xs font-medium bg-emerald-600 text-white"
 > { playing ? "Пауза" : "▶ Пуск" }
 </button>
 <button onClick={() => { setPlaying(false); setPlaying(true); }}
 className="px-3 py-1.5 rounded-lg text-xs font-medium bg-slate-700 text-white disabled:opacity-50"
 > Стоп
 </button>

 Шаг { idx + 1 } из { steps.length } • поворотов

 { step?.caseLabel && (
 <span className="px-2.5 py-1 rounded-full text-xs font-medium"
 > { step.caseLabel }

) }
 </div>
) }
}
```

```

 {nodes.map((n) => {
 const isAimNode = step?.aimEdge?.includes(n)
 return (
 <g
 key={`n-${n.key}-${treeVers}`}
 transform={`translate(${n.px}, ${n.py})`}
 onClick={() => {
 if (playing) return;
 const has = inorderKeys(baseRef.current)
 if (has) runKey(n.key);
 }}
 className="cursor-pointer"
 >
 <circle
 r={isAimNode ? 24 : 20}
 fill={stepColor(n.key)}
 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="rgba(0,0,0,0.1)" fontSize="13px">
 {n.key}
 </text>
 </g>
);
 })}
 </svg>

```

```

 /* Лог шага + механика */
 <div className="w-full flex flex-col lg:flex-row" >
 <div className="flex-1 bg-slate-950/80 p-4 rounded" >
 <p className="font-semibold text-amber-400 mb-2">
 {step?.kind === "aim" ? "Прицел:" : step?.kind === "search" ? "Спуск:" : "Старт:"}
 </p>
 <p className="min-h-[40px] flex items-center">
 ельности: прицел на ребро, поворот, перееха
 </p>
 </div>
 </div>

```

```

))
 <p className="text-xs text-slate-500 mt-3">Од
 щений дешевлеет.</p>
 </div>
 <div className="w-full lg:w-1/2 bg-slate-900/60"
 <h4 className="text-sm font-bold text-slate-2
 <div className="bg-slate-950/80 rounded-lg p-
 {CODE_LINES.map((l) => {
 const active =
 (step?.caseLabel === "Zig" && (l.line ===
 (step?.caseLabel === "Zig-Zig" && step.
 7 : step?.caseLabel === "Zig-Zig" && l.line
 (step?.caseLabel === "Zig-Zag" && step.
 10 : step?.caseLabel === "Zig-Zag" && l.line
 return (
 <div key={l.line} className={`py-1 px-2
 ber-400" : "text-slate-400"}`>
 <span className="text-slate-600 selec
 {l.text}
 </div>
);
 })}
 </div>
</div>
</div>
</div>
);
};

export default SplayWalkViz;

```

ФАЙЛ 63/81: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallback } from 'react';
```

```
const addLog = (msg: string) => setLog(prev => [msg,

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESET
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id === newP
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
```

```

 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
 });

const topIndex = dirtyStack.length - 1;

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950/40 p-6">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">Правило</h3>
 <p className="text-xs text-slate-300 mt-1 leading-tight">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно самую верхнюю.
 Попробуй вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white text-sm font-medium
 rounded-lg transition-all cursor-pointer flex items-center justify-center">
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white text-sm font-medium
 rounded-lg transition-all cursor-pointer flex items-center justify-center">
 Вытащить чистую (POP)
 </button>
 </div>
 </div>
);

```

```

"></div>
 <div className="absolute w-5 h-5 rounded-
"></div>
 <div className="absolute text-4xl anim-cr
</div>
})

{dirtyStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-
 <p className="text-slate-500 text-xs mt-1
 </div>
) : (
 <div className={`flex-1 flex flex-col justi

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}
 >
 <span className="text-xs font-mono text
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-p
 </div>

 {/* Тарелки (отрисовываем в обратном поряд
 {dirtyStack.slice().reverse().map((plate,
 const idx = dirtyStack.length - 1 - rev
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded

```

```

 <div className="flex-1 flex flex-col item

 <p className="text-xs font-bold font-mo
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-2 w-full
 {cleanRack.map(plate => (
 <div
 key={plate.id}
 className="w-full h-8 rounded-full
items-center justify-center gap-2 opacity-1
 >
 {plate.i
 <span className="text-[10px] text-e
 </div>
)}}
 </div>
)}
 </div>

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r f
 </div>
 </div>

 {/* ЛОГ ДЕЙСТВИЙ */}
 <div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slat
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-blue-500
 {entry}
 </div>
)}}
)}
)}

```



```

 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 совпадений нет.` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
}
return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i

```

```

 if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
 else return generateZSteps(safePattern, safeText);
 }, [mode, safePattern, safeText]);

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { setStepIdx(0); }
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval(timer.current); };
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setStepIdx(stepIdx + 1); };
const prevStep = () => { setAutoPlay(false); setStepIdx(stepIdx - 1); };
const reset = () => { setAutoPlay(false); setStepIdx(0); };

const step = steps[stepIdx] || steps[0];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <div className="flex bg-slate-900 border" style={borderStyle}>

```

```

 if (mode === "kmp") {
 if (step.highlight?.include
 borderColor = "border-a
 bgColor = step.match ===
-900/40");
 }
 }

 // Z Logic
 if (mode === "z") {
 if (index >= step.l && index
 bgColor = "bg-emerald-900
 borderColor = "border-em
 }
 if (step.zTarget === index
 borderColor = "border-a
 bgColor = step.match ===
-900/40");
 }
}

return {
 <div key={index} className=
 {/* L/R markers for Z */
 {mode === "z" && step.l
 <div className="abs
 }}
 {mode === "z" && step.r
 <div className="abs
 }}

 {/* Index */}
 <span className={`text-

 {/* Char Box */}
 <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
 {char}

```

```

 })
 {mode === "z" && step.z}
 <div className="abs
 <span className=
 </div>
 })

 {/* Found flash effect
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 </div>
)
 }
}
}
</div>
</div>

<div className="flex flex-col sm:flex-row g
 {/* Controls */}
 <div className="flex bg-slate-900 borde
 <button onClick={prevStep} disabled
rounded-lg disabled:opacity-30 disabled:hov
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={playPause} classNam
x items-center justify-center min-w-[80px]"
 {autoplay ? <><Pause size={16} <
 </button>
 <button onClick={nextStep} disabled
r:bg-slate-800 rounded-lg disabled:opacity-
 <SkipForward strokeWidth={3} si
 </button>
 <button onClick={reset} className="

```

```

 <pre className="p-4 text-xs font-mono text-gray-800">
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
}`}

 </pre>
 </div>
</div>
);
}

```

ФАЙЛ 65/81: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";

interface TNode {
 id: number;
}

```

```

const [currentStepIdx, setCurrentStepIdx] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];

 const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
 };

 recordStep(
 "Начало топологической сортировки. Используем кла",
 null,
);

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, () => []);
 dagEdges.forEach((e) => adj[e.u].push(e.v));

 const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} ("${dagNodes[u].name",
 u,
);
 };

```

```

 setCurrentStepIdx(0);
 }, []));

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

return {

```

```

 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
 </marker>
 <marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
 </marker>
</defs>

{/* Edges */}
{dagEdges.map(({edge, idx}) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v));

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}

```



```

 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-100"
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" ").join(" ")}
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join(" ")}
 </text>
 </g>
);
}
}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 text-white font-mono text-sm" style={{
 overflow-y: auto">

```

```

 </div>
 <div className="flex items-center gap-2">

 Черные (готово)

 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-200 text-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-200 text-white"
 >
 Вперед
 </button>
 </div>
</div>
</div>
</div>
</div>
);
};

```

```

}

export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
 { key: 1, grade: -4 },
];

/** Порядок ввода (как в черновике) – намеренно НЕ отсортирован */
export const INPUT_ORDER: Pair[] = [
 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 1, grade: -4 },
 { key: 7, grade: 2 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 12, grade: 0 },
];

// (3,5) дубль ключа/приоритета – уберём: ввод без повторов
export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p, i) => !INPUT_ORDER.some((p2, j) => j < i && p2.key === p.key));

const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => a.grade - b.grade);

/* ----- чистые алгоритмы (экспортируются для тестов) ----- */

export function insert(root: TNode | null, p: Pair): TNode {
 if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade, left: null, right: null };
 if (p.key <= root.key) root.left = insert(root.left, p);
 else root.right = insert(root.right, p);
 return root;
}

```

```

const trace: MergeResult["trace"] = [];
const go = (a: TNode | null, b: TNode | null): TNode
 if (!a || !b) {
 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null, note: "Обе ветви пустые, возвращаем целое целиком." });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: "Выбираем левую ветвь, так как ее правое поддеревое с (${b.key}, ${b.grade}) не содержит элементов с ключом больше ${a.key}." });
 a.right = go(a.right, b);
 return a;
 }
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: "Выбираем правую ветвь, так как ее левое поддеревое с (${a.key}, ${a.grade}) не содержит элементов с ключом больше ${b.key}." });
 b.left = go(a, b.left);
 return b;
};
return { root: go(a, b), trace };
}

export function eraseNode(root: TNode | null, key: number): TNode | null {
 const trace: { atId: NodeId | null; note: string }[] = [];
 const go = (t: TNode | null): TNode | null => {
 if (!t) return null;
 if (key < t.key) {
 trace.push({ atId: t.id, note: `${key} < ${t.key}` });
 t.left = go(t.left);
 return t;
 }
 if (key > t.key) {
 trace.push({ atId: t.id, note: `${key} > ${t.key}` });
 t.right = go(t.right);
 return t;
 }
 trace.push({ atId: t.id, note: `Нашли (${t.key}, ${t.grade})` });
 const m = mergeTree(t.left, t.right);
 trace.push(...m.trace.map(s => ({ atId: s.chosenId, note: s.note })));
 return m.root;
 };
 const root = go(root);
 return { root, trace };
}

```

```

 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
};

const treeEdges = (root: TNode | null): { from: NodeId;
const es: { from: NodeId; to: NodeId }[] = [];
const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ from: n.id, to: n.left.id });
 if (n.right) { es.push({ from: n.id, to: n.right.id
 };
 walk(root);
 return es;
};

interface TreeViewProps {
 root: TNode | null;
 width?: number;
 height?: number;
 nodeColor?: (id: NodeId) => string;
 nodeStroke?: (id: NodeId) => string;
 edgeColor?: (from: NodeId, to: NodeId) => string;
 edgeDash?: (from: NodeId, to: NodeId) => boolean;
 pulseId?: NodeId | null;
 subtitle?: string;
}

const TreeView: React.FC<TreeViewProps> = ({ root, width
 const pos = useMemo(() => layoutTree(root), [root]);
 const nodes = useMemo(() => [...pos.entries()].map(([
 const edges = useMemo(() => treeEdges(root), [root]);
 const autoW = Math.max(420, ...nodes.map((n) => n.px
 const autoH = Math.max(260, ...nodes.map((n) => n.py
 const w = width ?? autoW;
 const h = height ?? autoH;
 const byId = (id: NodeId) => nodes.find((n) => n.id =

```

```

 return [a, b];
 };

 /* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 60) / 10);
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H - 60) / 10);

interface PlaneProps {
 points: Pair[];
 drawnEdges: { a: Pair; b: Pair }[];
 pending: Pair | null;
 interactive?: boolean;
 selected?: Pair | null;
 cursor?: { x: number; y: number } | null;
 badEdge?: { a: Pair; b: Pair } | null;
 onPointDown?: (p: Pair) => void;
 onPointUp?: (p: Pair) => void;
 onMove?: (e: React.PointerEvent<SVGSVGElement>) => void;
}

const Plane: React.FC<PlaneProps> = ({ points, drawnEdges, pending,
 }) => {
 const byPair = (p: Pair) => ({ x: PX(p.key), y: PY(p.grade) });
 return (
 <svg
 viewBox={`0 0 ${PL_W} ${PL_H}`}
 className={`w-full ${interactive ? "touch-none select-none" : ""}`}
 onPointerMove={onMove}
 style={interactive ? { cursor: "crosshair" } : undefined}
 >
 <defs>
 <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" />
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 46} />
 </svg>
);
};

```



```

const heap = edges.every((e) => e.a.grade > e.b.grade)
const oneParent = new Set(edges.map((e) => gpid(e.b)))
const acyclic = !edges.some((e) => reaches(edges, e.a, e.b))
const bst = forestIsBst(edges, all);
const roots = all.filter((p) => !edges.some((e) => gp(e.a) === p));
const complete = edges.length === all.length - 1 && roots.length === 1;
return { heap, oneParent, acyclic, bst, complete, win };
}

```

/\*\* Попытка хода: почему нельзя – или ОК. \*/

```

export function tryConnect(edges: GEdge[], a: Pair, b: Pair): boolean {
 if (sameP(a, b)) return { ok: false, reason: "Это одна и та же точка" };
 if (a.grade === b.grade)
 return { ok: false, reason: "«!» равны – «кто выше?»" };
 const parent = a.grade > b.grade ? a : b;
 const child = a.grade > b.grade ? b : a;
 if (parentOf(edges, gpid(child)))
 return { ok: false, reason: `У точки (${child.key}) уже есть родительская` };
 if (reaches(edges, parent, child))
 return { ok: false, reason: "Цикл: линия замкнёт саму себя" };
 const next = [...edges, { a: parent, b: child }];
 if (!forestIsBst(next, POINTS))
 return { ok: false, reason: "Линия вниз по y допустима только если не образует цикла" };
 return { ok: true };
}

```

/\* ===== РЕЖИМ BUILD ===== \*/

```

type BuildPhase = "sort" | "connect" | "game";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";

```

```

const parsePairs = (raw: string): { pairs: Pair[]; errors: string[] } => {
 const pairs: Pair[] = [];
 const errors: string[] = [];
 const seen = new Set<string>();
 const chunks = raw.split(/[,;\n]+/).map((c) => c.trim());
 for (let i = 0; i < chunks.length; i++) {
 const [key, value] = chunks[i].split(":");
 if (!key || !value) errors.push(`Ошибка синтаксиса строки ${i + 1}`);
 else {
 const pair = { key, value };
 if (seen.has(key)) errors.push(`Повторяющаяся точка ${key} строки ${i + 1}`);
 else pairs.push(pair);
 seen.add(key);
 }
 }
 return { pairs, errors };
}

```



```

 return { ok: false, reason: "такая линия уже пров
 }
 if (x.grade === y.grade) return { ok: false, reason:
 " };
 const [parent, child] = x.grade > y.grade ? [x, y] :
 if (edges.some((e) => pid(e.b) === pid(child)))
 return { ok: false, reason: `у точки (${child.key});
 // цикл: родитель уже сидит в компоненте ребёнка (реб
 const childComp = new Set<string>([pid(child)]);
 let grew = true;
 while (grew) {
 grew = false;
 for (const e of edges) {
 if (childComp.has(pid(e.b)) && !childComp.has(pid
 childComp.add(pid(e.a));
 grew = true;
 }
 }
 }
 if (childComp.has(pid(parent))) return { ok: false, r
 return {
 ok: true,
 edge: { a: parent, b: child },
 reason:
 x.grade > y.grade ? undefined : "линия в treap вс
 };
}

```

```

export function checkGameState(
 points: Pair[],
 edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected:
 const n = points.length;
 if (n < 2) return { heapOk: true, singleParent: true,
 const heapOk = edges.every((e) => e.a.grade > e.b.gra
 const parentCount = new Map<string, number>();
 for (const e of edges) parentCount.set(pid(e.b), (par
 const singleParent = [...parentCount.values()].every(

```

```

 let right: Pair | null = null;
 if (sorted.length === 2) {
 if (sorted[0].key === sorted[1].key || sorted[0].key > sorted[1].key) {
 bstOk = false; // двое детей с равным x или с
 return;
 }
 left = sorted[0];
 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяет
 if (sorted[0].key <= p.key) left = sorted[0];
 else right = sorted[0];
 }
 if (left) walk(left);
 inorder.push(p.key);
 if (right) walk(right);
 };
 walk(roots[0]);
 if (seen.size !== n) bstOk = false;
 for (let i = 1; i < inorder.length; i++)
 if (inorder[i] < inorder[i - 1]) bstOk = false;
}
return { heapOk, singleParent, connected, countOk, bstOk };
}

```

/\*\* Дерево из проведённых линий – для показа результата

```

export function edgesToTree(edges: GameEdge[]): TNode | null {
 const parentOf = new Map<string, GameEdge>();
 for (const e of edges) parentOf.set(pid(e.b), e);
 const roots = edges.filter((e) => !parentOf.has(pid(e.b)));
 if (!roots.length && edges.length) return null;
 const nodes = new Map<string, TNode>();
 const make = (p: Pair): TNode => ({
 id: pid(p),
 key: p.key,
 grade: p.grade,
 left: null,
 right: null,
 });
 for (const e of edges) {
 const p = parentOf.get(pid(e.b));
 if (p) {
 const node = make(p);
 if (e.a < e.b) node.left = make(e);
 else node.right = make(e);
 nodes.set(pid(p), node);
 } else {
 const node = make(e);
 nodes.set(pid(e.b), node);
 }
 }
 return roots.length ? make(roots[0]) : null;
}

```

```

 if (!node) return;
 if (node.left) { es.push({ a: { key: node.key, gr
 left}); }
 if (node.right) { es.push({ a: { key: node.key, g
 de.right}); }
 };
 walk(userTree);
 return es;
}, [userTree]);
const leftRightEdges = useMemo(() => [...canonEdges].

useEffect(() => {
 if (!playing || phase === "game") return;
 const t = setTimeout(() => {
 if (phase === "sort") {
 if (algo === "none") { setPlaying(false); return;
 if (sortIdx < 3) setSortIdx(sortIdx + 1);
 else setPlaying(false);
 } else {
 const total = order === "y-desc" ? canonEdges.l
 if (connIdx < total) setConnIdx(connIdx + 1);
 else setPlaying(false);
 }
 }, 900);
 return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, ca

const startConnect = (ord: ConnectOrder) => {
 setOrder(ord);
 setPhase("connect");
 setConnIdx(0);
 setPlaying(true);
};

const comparisons = algo === "quicksort" ? (n > 1 ? M
const connEdgesAll = order === "y-desc" ? canonEdges
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && con

```

```

 };
 const pointDown = (p: Pair) => {
 if (phase !== "game") return;
 if (!sel) {
 setSel(p);
 setGameMsg(`Выбрана (${p.key}; ${p.grade}) – теперь...`);
 } else if (sel.key === p.key && sel.grade === p.grade) {
 setSel(null);
 setGameMsg(null);
 } else {
 attemptConnect(sel, p);
 setSel(null);
 }
 };
 const pointUp = (p: Pair) => {
 if (phase !== "game" || !sel) return;
 if (sel.key === p.key && sel.grade === p.grade) return;
 attemptConnect(sel, p);
 setSel(null);
 setCursor(null);
 };
 const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
 if (!sel) return;
 const rect = (e.currentTarget as SVGSVGElement).getBBox();
 setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100 });
 };

 const undoEdge = () => {
 setGameEdges((prev) => prev.slice(0, -1));
 setGameMsg(null);
 };

 const showSolution = () => {
 setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
 setHintUsed(true);
 setGameMsg("Решение показано – собери сам в следующий раз!");
 };

```

```

<div className="flex items-center gap-2">
 {phase === "sort" ? (
 <>

 {["quicksort", "counting"] as const }.map((a) => {
 <button
 key={a}
 onClick={() => { setAlgo(a); setSort(a);
 Math.round(n * Math.log2(n)) : 0 } сравнений
 disabled={n < 2}
 className={ `px-3 py-1.5 rounded-lg text-slate-400
 bg-emerald-600 text-white" : "bg-slate-700 text-slate-400` }
 >
 {a === "quicksort" ? "quicksort (сравнения)" : "counting"}
 </button>
) : (
 <>

 <button
 onClick={() => startConnect("y-desc")}
 className={ `px-3 py-1.5 rounded-lg text-slate-400
 bg-emerald-600 text-white" : "bg-slate-700 text-slate-400` }
 >
 по y ↓ (алгоритм)
 </button>
 <button
 onClick={() => startConnect("left-right")}
 className={ `px-3 py-1.5 rounded-lg text-slate-400
 "bg-emerald-600 text-white" : "bg-slate-700 text-slate-400` }
 >
 слева направо
 </button>
 <button
 onClick={startGame}
 disabled={n < 2}
 className={ `px-3 py-1.5 rounded-lg text-slate-400

```

```

 gameWon ? (
 <> Готово! {hintUsed ? "Решение подсказано: линии вниз по y, обход по x отсортировано"} : gameMsg ? (
 <>△ {gameMsg}</>
) : (
 <>Кликни точку-родителя (выше по «!»), за
)
) : connDone ? (
 <> Проведено {connEdges.length} линий в по
 : точки прибиты, дерево одно. А теперь
) : (
 <>Линий проведено: {connIdx} из {connEdgesA
)}
 </div>
 </div>

```

```

 { /* Чек-лист инвариантов */ }
 <div className={`rounded-xl p-4 border transition
 er-slate-700 bg-slate-900/40`} `>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {phase === "game" ? (
 <>
 {verdict.heapOk ? " " : " "} каждое р
 {verdict.singleParent ? " " : " "} у
 {verdict.connected ? " " : " "} все т
 {verdict.countOk ? " " : " "} линий р
 {verdict.bstOk ? " " : " "} обход сле
 </>
) : (
 <>
 {connDone ? " " : " "} каждое ребро в
 {connDone ? " " : " "} обход слева-на
 {connDone ? " " : " "} линий ровно
 </>
)}

 </div>

```

```

const split = useMemo(() => splitTree(buildTree(POINT:
const steps = split.trace;
const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;

const inL = new Set<NodeId>();
const inR = new Set<NodeId>();
steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? in

const lTree = split.l;
const rTree = split.r;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Разрез
 <select value={x0} onChange={(e) => { setX0(Num
 -700 rounded-lg px-3 py-1.5 text-sm font-mono
 {[2, 3, 6, 7, 9, 12, 45].map((k) => <option k
 </select>
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:o
 шаг {i
 </div>

 <div className="bg-slate-900/40 rounded-xl p-4 bo
 <h4 className="text-sm font-bold text-slate-300
 , розовые → R
 <TreeView
 root={fullTree}
 nodeColor={(id) => (inL.has(id) ? "#1e3a8a" :
 nodeStroke={(id) => (inL.has(id) ? "#60a5fa"
 subtitle={done ? "Каждый узел покрашен ровно
 </>
 </div>

```

```

<div className="flex items-center gap-3">
 <button onClick={() => setStepIdx(Math.max(0, i-1))}
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(steps-1, i+1))}
 font-bold bg-indigo-600 text-white disabled:opacity-30">
 Шаг {i+1} из {steps}
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-3">
 <div className="bg-sky-950/20 rounded-xl p-3 border border-slate-700">
 <h5 className="text-sm font-bold text-sky-300"> Левое дерево
 <TreeView root={l} width={480} height={220} nodeRenderer={nodeRenderer} />
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 border border-slate-700">
 <h5 className="text-sm font-bold text-rose-300"> Правое дерево
 <TreeView root={r} width={480} height={220} nodeRenderer={nodeRenderer} />
 </div>
</div>

<div className="text-xs text-slate-300 bg-slate-900 p-3 border border-slate-700">
 {cur?.chosenId ? (
 <div className="flex items-center gap-2">
 📄 {cur.note}</div>
) : (
 <div>Готово: {done ? "деревья склеены обратно в одно" : "ожидание"}</div>
)}
</div>

{done && (
 <div className="bg-emerald-950/20 rounded-xl p-3 border border-slate-700">
 <h5 className="text-sm font-bold text-emerald-300"> Результат
 <TreeView root={merged.root} subtitle="merge(l, r)" />
 </div>
)}

<div className="text-xs text-slate-400 bg-slate-900 p-3 border border-slate-700">
 Грамотно про «слияние»: у корней сравниваются значения
 в (то, что сохраняет порядок x) отправляется в левое дерево,

```



```

 {!done ? (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <TreeView
 root={fullTree}
 nodeColor={(id) => (pathSet.has(id) ? "#783"
 nodeStroke={(id) => (pathSet.has(id) ? "#f5
 subtitle="Куча сама умеет удалять только ве
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={res.root} subtitle="Валидность
 <ul className="mt-2 space-y-1 text-sm">
 обход слева-направо по-прежнему сорти
 каждое ребро вниз по у (куча целая) –

 </div>
))

 <div className="text-xs text-slate-300 bg-slate-9
 {steps[idx]?.note}
 </div>

 <div className="text-xs text-slate-400 bg-slate-9
 Ответ на «убрать можно только самый верхний
 ягивает merge двух детей. Удаляется любая<
 </div>
 </div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("h
 // Полное дерево из 7 узлов (куча): индексы 1..7
 const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];

```

```

 → $2i/2i+1$ попадает в узел.</p>
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-rose-300">
 >
 <TreeView root={full} subtitle="Попробуй «пос
 " />
 <div className="flex gap-2 overflow-x-auto mt-2">
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]}
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 rounded-slate-700` : "bg-rose-950/40 border-rose-950">
 <div className="text-slate-500 text-sm">
 {id ? <div className="text-white font-sm">
 </div>
 } :
 </div>
 </div>
);
 </div>
 <p className="text-xs text-rose-300 mt-2">Смотри, что левая ветка ушла на 4 уровня и заняла два уровня указателями (left/right), а не арифметикой.</p>
 </div>
)}
}

<div className="text-xs text-slate-400 bg-slate-900/40 p-2">
 Грабля «дети в массиве по $2k/2k+1$ »: форматированным: слои дырявые, и «адрес» ребёнка посчитан
</div>
</div>
);
};

/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) ===== */

const SearchMode: React.FC = () => {
 // Подобрено так, что бинарный поиск ГАРАНТИРОВАННО

```

```

 {stage < 2 && (
 <button onClick={() => { setStage(stage === 0
 digo-600 text-white">
 {stage === 0 ? "Найти у для x=7 бинарным по
 </button>
)}
 {stage === 2 && (
 <button onClick={() => setBidx(Math.min(bsSte
 -1.5 rounded-lg text-xs font-bold bg-indigo
 Шаг поиска →
 </button>
)}
 <button onClick={() => { setStage(0); setBidx(0
 Сброс</button>
 </div>

 <div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p,
 const highlight = stage === 1 && i === wrongM
 const inBs = stage === 2 && (() => { const s
 const isMid = stage === 2 && (() => { const s
 const isFound = stage === 2 && bidx >= bsStep
 return (
 <div key={`_${p.key}:${p.grade}-${i}`} class
 nsition-all ${highlight ? "border-rose-500
 r-amber-500 bg-amber-950/30" : inBs ? "bord
 <div className="text-white font-bold">{p.
 <div className="text-slate-400 text-[10px
 </div>
);
)}
)}
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
 {stage === 0 && <>Массив пар не отсортирован
 а беспорядок.</>}
 {stage === 1 && <> Бинарный поиск посмотрел на
 ю половину... в которой лежала цель. На

```

```

const res = tryConnect(edges, sel, p);
if (res.ok) {
 const parent = sel.grade > p.grade ? sel : p;
 const child = sel.grade > p.grade ? p : sel;
 setEdges([...edges, { a: parent, b: child }]);
 setMsg(`  Линия (${parent.key}; ${parent.grade})`);
 setSel(null);
} else {
 setMsg("x " + res.reason);
 setSel(null);
}
};

const win = v.win;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setEdges(edges.slice(0,
 rounded-lg text-xs font-bold bg-slate-700 tex
 <button onClick={() => { setEdges([]); setSel(n
 3 py-1.5 rounded-lg text-xs font-bold bg-slate
 <button onClick={() => { setEdges(CANON_GAME.ma
); }} className="px-3 py-1.5 rounded-lg text-
 линий:
 </div>

 <div className="bg-emerald-950/20 rounded-xl p-4 l
 <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} classl
 <defs>
 <pattern id="gm-grid" width="14" height="14"
 <path d="M 14 0 L 0 0 0 14" fill="none" s
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height=
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={P
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y

```

```

 {v.asyclic ? " " : " "} циклов нет
 {v.bst ? " " : " "} обход слева-направо —
 {v.complete ? " " : " "} линий $n-1 = \{POI$

 {win && {
 <p className="mt-3 text-sm font-bold text-eme
 ения просто не существует.</p>
 }}
</div>

 <div className="text-xs text-slate-300 bg-slate-90
</div>
);
};

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "lay

const TABS: { id: Tab; label: string }[] = [
 { id: "build", label: "Собрать" },
 { id: "split", label: "Split $\times$ " },
 { id: "merge", label: "Merge " },
 { id: "erase", label: "Erase " },
 { id: "layers", label: "Миф: $2k/2k+1$ " },
 { id: "search", label: "Поиск $\neq$ сортировка" },
 { id: "game", label: "Игра " },
];

export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl bor
 <div className="flex flex-wrap items-center justifi
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-whi


```

```

import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import SplayWalkViz from "./SplayWalkViz";
import { SplayRotationSandbox } from "./SplayRotationSandbox";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimationWidget";
import ChapterImage from "./ChapterImage";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

```

```

treap: {
 title: "Treap: плоскость → дерево → Split/Merge/Erge",
 hint: "7 вкладок: Собрать (сортировка + соединение) →

 собери дерево мышкой, судья-инварианты объясняет к",
 Component: TreapBuildViz,
},
"prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число — увидите, из чего оно сло",
 Component: PrefixSumViz,
},
"dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
},
"splay-tree": {
 title: "Splay: демо на большом дереве → собери сам",
 hint: "Сначала демо: кликни узел — он поднимется в

 ца: подними узел сам, порядок сверится со splay.",
 Component: () => (
 <div className="space-y-10">
 <SplayWalkViz />
 <p className="flex items-start gap-2 text-[12px]

 px-3 py-2">

 Разобрался, как ходит один поворот? Ниж
 ок со splay.
 </p>
 <SplayRotationSandbox />
 </div>
),
},
"splay-rotations": {
 title: "Zig / Zig-Zig / Zig-Zag: демо и песочница",
 hint: "Каждый одиночный поворот — отдельный шаг с м
 Component: () => (
 <div className="space-y-10">

```

```

 },
 },
 "johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonAlgo },
 "mst-kruskal": { title: "Краскал и DSU", Component: KruskalSimulator },
 "mst-prima": { title: "Прим", Component: KruskalSimulator },
 "mst-boruvka": { title: "Борувка", Component: KruskalSimulator },
 "string-kmp": { title: "Префикс-функция (КМП)", Component: KMP },
 "string-z-func": { title: "Z-функция", Component: ZFunction },
 "aho-corasick": {
 title: "Ахо-Корасик",
 hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
 Component: () => (
 <div className="space-y-10">
 <WaterfallAnimationWidget />
 <SalmonAutomatonWidget />
 </div>
),
 },
 intro: { title: "Мнемокарточки", Component: MnemonicCards },
 "everyday-basics": {
 title: "Бытовой тренажёр: стек, очередь, куча",
 hint: "Тарелки, очередь в столовой и мешок гипотез",
 Component: Simulator,
 },
};

```

```
export const getViz = (id?: string): VizEntry | undefined;
```

ФАЙЛ 68/81: src/components/WaterfallAnimationWidget.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

```

```
// Структура заранее подготовленного красивого бора для
```



```

5: { S: 6 },
6: {},
7: { H: 8 },
8: { E: 9 },
9: {},
};

export const WaterfallAnimationWidget: React.FC = () => {
 // Переключатель режимов: 'training' (Полный BFS обход)
 const [activeMode, setActiveMode] = useState<'training' | 'search'>('training')

 // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
 const [textToScan, setTextToScan] = useState<string>('')
 const [currentIndex, setCurrentIndex] = useState<number>(0)
 const [currentNode, setCurrentNode] = useState<number>(0)
 const [isSearchDone, setIsSearchDone] = useState<boolean>(false)
 const [searchLog, setSearchLog] = useState<string>('')
 'Режим поиска: Введите текст и жмите «Следующий шаг»'
);
 const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>({ from: 0, to: 0 })
 const [foundMatches, setFoundMatches] = useState<{ index: number; path: string }[]>([])
 const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0)

 // === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
 // Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
 const [trainStep, setTrainStep] = useState<number>(0)

 const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/[^А-Яа-я0-9]/g, '')
 setTextToScan(clean)
 resetSearchSimulation()
 };

 const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся на базу.')
 }
}

```

```

 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${orig
 ружок по fail-ссылке в #${curr} (${WATERFALL_
 . `;
 setJumpPath({ from: originalCurr, to: nextNode,
 }
 curr = nextNode;
}

const currentMatches: { index: number; word: string
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
 if (WATERFALL_NODES[temp].is_terminal) {
 WATERFALL_NODES[temp].words.forEach((word) :
 currentMatches.push({ index: currentInd
 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWords
 setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMat
}
};

```

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В

```

 а-разведчик плывет в ROOT и ищет ветку по "E".
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'E',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 5,
 title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
 desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
 H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'I',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 8,
 title: 'Шаг 5 (Depth 2): Вершина #8 (SH) – КАК СТА
 desc: ' ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
). Рыба плывет в ROOT и ищет ветку "H". У корня
 суффикс "H"!',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
 е подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep]
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainStep]

return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border" >
 { /* Шапка и переключатель режимов */ }
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Анимация водопада: Полное обучение автомата
 </h4>
 <p className="text-sm text-slate-400" >
 Визуализация пошагового построения всех функций
 </p>
 </div>
 </div>
 </div>

 { /* Переключатель режимов */ }
 <div className="bg-slate-900 p-1.5 rounded-xl border" >
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p-2
 ${activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'}
 `}
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корня)
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p-2
 ${activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg shadow-blue-500'
 : 'text-slate-400 hover:text-white'}
 `}
 >
 <GitBranch className="w-4 h-4" />
 Режим 2: Поиск по дереву (DFS)
 </button>
 </div>
)
)

```

```

 }}}
 </div>
 </div>

 <div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg
ion-all"
 >
 Назад
 </button>
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === TRAINING_STEPS_
 className="flex-1 bg-indigo-600 hover:b
ition-all shadow-lg shadow-indigo-900/50"
 >
 {trainStep === TRAINING_STEPS_INFO.leng
 </button>
 </div>
 </div>

 {/* Блок с кодом построения fail-ссылки и опи
 <div className="lg:col-span-2 bg-slate-900/90
 >
 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 ❌ Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex ite

```



```

border-blue-500 text-blue-300 font-bold' :
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 &&
инициализация}
</div>
<div className={`p-1.5 rounded flex item
border-rose-500 text-rose-300 font-bold an
 2. while (curr !== root && !trie
 {activeSearchCodeLine === 2 &&
рыжок по fail!}
</div>
<div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold' :
 3. curr = trie[curr].get(char)
 {activeSearchCodeLine === 3 &&
ем по течению}
</div>
<div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
 4. if (is_terminal[curr]) repor
 {activeSearchCodeLine === 4 &&
овпадение!}
</div>
</div>
</div>

```

```

<div>
 <h5 className="text-slate-300 font-bold m
 <AlertCircle className="w-3.5 h-3.5 tex
 </h5>
 <div className="bg-slate-800 p-3.5 rounde
tems-center">
 {searchLog}
 </div>
</div>
</div>
</div>

```

```

 })

```

```

].map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="14" height="24" />
 />
 <text x="32" y={level.y + 4} fill="#94a3b8" font-size="12">
 {level.label}
 </text>
 </g>
)})
}

```

```

 /* Рендер прямых переходов (потоков воды) */
 Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
 const fromNode = WATERFALL_NODES[Number(fromId)];
 return Object.entries(children).map(([childId, child]) => {
 const toNode = WATERFALL_NODES[toId];

```

```

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' &&
 childId === toId;
 });
 });
 type === 'normal';

```

```

 // Подсветка в режиме обучения (когда ищем путь)
 let isTrainingExamined = activeMode === 'training' &&
 childId === toId;
 let isMatchBranch = isTrainingExamined &&
 childId === toId;

```

```

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`}>
 /* Линия течения воды */
 <line

```





```

 textAnchor="middle"
 >
 {node.label}
 </text>

 {/* Метка искомой вершины в режиме об
 {isTargetChild && {
 <g transform={`translate(${node.x},
 <rect x="-35" y="-12" width="70"
 <text x="0" y="2" fontSize="10" f
 ИЩЕМ FAIL
 </text>
 </g>
 }}

 {/* Индикатор словарных слов */}
 {hasWords && {
 <g transform={`translate(${node.x +
 <circle cx="0" cy="0" r="9" fill=
 <text x="0" y="3" fontSize="10" f
 *
 </text>
 </g>
 }}
 </g>
 });
 }}}

 {/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}
 <g>
 {/* Круг-подложка под рыбу */}
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5
 stroke="#ffffff"
 strokeWidth="2"

```

```

 <CheckCircle2 className="w-4 h-4 text-
 {match.word}
 <span className="text-[10px] bg-emera
 Индекс {match.index}

 </div>
)})
 </div>
})
</div>
});
};
```

## РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 69/81: src/components/visualizers/PrefixSumViz.tsx  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМ:

```
import React, { useМemo, useState } from "react";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * * в 1D наводим на P[i] – подсвечивается кусок массива
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения-исключения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];
```

```

/* ----- ID -----
const PrefixID: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[0] + v],
 []);

 /** Что сейчас под курсором: префикс P[i] или элемент a[i]
 const [hover, setHover] = useState<{ kind: "pref" | "a"; i: number }>({ kind: "pref", i: 0 });
 const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });

 const covered =
 hover?.kind === "pref"
 ? { from: 0, to: hover.i - 1 }
 : hover?.kind === "a"
 ? { from: hover.i, to: hover.i }
 : null;

 const sum = pref[range.r + 1] - pref[range.l];

 return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
 <div className="inline-block min-w-full">
 /* индексы */
 <div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map((_, i) => (
 <div key={i} className={` ${cellBase} !h-5`}>
 {i}
 </div>
))}
 </div>

 /* массив a */
 <div className="flex gap-1.5 items-center mb-1">
 <div className="w-[46px] shrink-0 text-right">
 {A1.map((v, i) => {

```

```

 : isR
 ? "bg-emerald-700 text-white"
 : isL
 ? "bg-rose-700 text-white"
 : "bg-slate-900 text-slate-300"
 }` }
 title={`P[${i}] = сумма a[0..${i} - 1]
 >
 {v}
 </div>
);
 }}}
</div>
</div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border border-slate-700" >
 {hover?.kind === "pref" ? {
 hover.i === 0 ? {
 <p className="text-slate-300">
 <b className="text-emerald-400">P[0] = 0 —
 отрезка, начинающегося с нуля.
 </p>
 } : {
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 — это сумма всего, что набежало от начала

 a[0..{hover.i} - 1] = {A1.slice(0, hover.i)}

 </p>
 }
 }
 } : hover?.kind === "a" ? {
 <p className="text-slate-300">
 <b className="text-indigo-400">

```

```
const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array(
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {
 p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i]
 }
 }
), [n, m]);
```

```
const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });
```

```
const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;
```

```
const cornerOf = (i: number, j: number) => {
 if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
 if (i === rect.r1 && j === rect.c2 + 1) return "B";
 if (i === rect.r2 + 1 && j === rect.c1) return "C";
 if (i === rect.r1 && j === rect.c1) return "D";
 return null;
};
```

```
return (
 <div className="space-y-5">
 <div className="grid gap-5 lg:grid-cols-2">
 {/* Исходная матрица */}
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">Исходная матрица A2</div>
 <div className="inline-block">
 {A2.map((row, i) => (
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2
```

```

 return {
 <div
 key={j}
 onMouseEnter={() => setHover({ i,
 onMouseLeave={() => setHover(null
 className={` ${cellBase} cursor-he
 isHover ? "bg-amber-500 text-wh
 }`}
 >
 {v}
 {corner && !isHover && (
 <span className="absolute -top-
ht">
 {corner}

)}
 </div>
);
 })))
 </div>
)})
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border
 {hover ? (
 <p className="text-slate-300">
 <b className="text-amber-400">
 P[{hover.i}][{hover.j}] = {P[hover.i][hov
 {" "}
 – сумма всего прямоугольника от левого верх
 </p>
) : (
 <p className="text-slate-500">Наведите курсор
)}
</div>

```

```

for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j) - 1][j - 1]);
 }
}

```

// Data for 2D Matrix (Sparse Table 2D)

```

const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
 [25, 41, 16, 92, 9, 17, 56, 31],
 [59, 18, 77, 24, 61, 82, 35, 12],
 [73, 8, 38, 85, 47, 95, 19, 58],
 [39, 81, 62, 28, 51, 42, 85, 14]
];

```

```

const ST2D: number[][][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);

```

```

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

```

```

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {

```



```

 se-600 text-white" : "bg-slate-800 text-sla
 >
 3. Запрос 2D (Формула)
 </button>
</div>

<AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
</AnimatePresence>
</div>
);
};

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number}>({ i: 0, j: 0 });

 // Total steps: we animate computing each element for
 const computeSteps: { i: number; j: number }[] = [];
 for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
 }

 const maxStep = computeSteps.length;
 const covered = hover ? { from: hover.i, to: hover.i + 1 } : null;

 return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>

```

```

 <div
 className={`flex items-center justify
w,44px)}] h-[clamp(26px,8vw,44px)] text-[cla
inCover ? "bg-indigo-500 text-white
 }`}
 >
 {v}
 </div>
 <span className="text-[9px] text-slate-1
 </div>
);
 }}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
 {hover && covered ? (
 <p className="text-slate-300">
 <b className="text-sky-400">
 ST[{hover.i}][{hover.j}] = {st1D[hover.i
 {" "}
 – это минимум на отрезке{" "}

 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\{hover.j\}}$ = {1 << hover.j}:{" "}

 min({arr1D.slice(covered.from, covered.

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое чи
 </p>
)}
</div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate

```



```

 </motion.svg>
)}
 </div>
);
}]]
</React.Fragment>
)}}
</div>
</div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-
 {step > 0 && step <= maxStep ? (
 (() => {
 const c = computeSteps[step - 1];
 const half = 1 << (c.j - 1);
 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(
 <span className="text-emerald-400 font-l
 ST[{c.i}][{c.j - 1}]

 ,
 <span className="text-amber-400 font-bo
 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1D

 {st1D[c.i + half][c.j - 1]}

) ={" "}
 <span className="text-indigo-400 font-b
 {st1D[c.i][c.j]}

```

```

 className={`px-4 py-2 rounded-md text-sm
t-slate-400 hover:text-white hover:bg-slate
>
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${s
</button>
)}}
</div>

```

```

<div className="flex flex-col items-center gap-
rollbar">
 {[...Array(k + 1)].map((_, i) => k - i).map((
 const stepL = 1 << step;
 const stepSize = 8 - stepL + 1;
 const isCurr = step === k;

 return (
 <React.Fragment key={step}>
 {idx > 0 && <div className="text-slate
 <div className={`flex flex-col items-c
 <h4 className={`font-bold mb-3 flex
 }>
 Матрица блоков {stepL}x{step
 <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
 </h4>
 <div className="max-w-full overflow-
 <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

 let highlightClass = isCurr ? '
/50 cursor-pointer' : 'bg-slate-800/50 text
 let zIndex = 'z-0';

 const isActive = !!activeCell &

```

```

 }
 }}
 className={`flex items-center
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp(
clamp(20px,5.2vw,30px)] text-[clamp(8px,2vw
>
 {ST2D[r][c][step][step]}
 </div>
);
 }}}
</div>
</div>
</div>
</React.Fragment>
)
 }}}
</div>
</div>

```

```

<div className="flex-1 min-w-0 flex flex-col gap-4"
 <div className="bg-slate-900 p-4 sm:p-6 rounded-2xl"
 <h3 className="text-lg font-bold text-slate-300"
 {k === 0 ? (
 <div className="text-slate-400 text-sm leading-6"
 <p className="mb-4">При <span className="font-
ы имеют размер <span className="font-bold text-slate-300"
 <div className="bg-[#0a0ffe] rounded-xl p-4"
 // Ба
 ST
 </div>
 <p className="mt-6 italic text-indigo-400"
 <Play size={14} className="fill-indigo-400"
 </p>
 </div>
) : (
 <div className="flex flex-col gap-4 animate-fadeIn"
 <div className="bg-[#0a0ffe] rounded-xl p-4"
 <div className="text-slate-300 shadow-inner overflow-x-auto">

```

```

 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#f43f5e]"></div> ST[{activeC
 <span className="text
tiveCell.r][activeCell.c][k-1][k-1]}
 </div>
 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#3b82f6]"></div> ST[{activeC
 <span className="text
tiveCell.r][activeCell.c + prevL][k-1][k-1]
 </div>
 <div className="flex item
 <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
 <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
 </div>
 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
)
 }}())}
</div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : (
 <div className="bg-slate-950/40 border

```

```

 {Array.from({length: 64}).map((_,
 const r = Math.floor(idx / 8);
 const c = idx % 8;
 const isInQuery = r >= r1 && r

 return (
 <div key={idx} className={`
y-center rounded text-[clamp(9px,2.4vw,12px
0 border-rose-500/50 border scale-105 z-10'
 {val2D[r][c]}
 </div>
)
 }}}

 <div
 className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
 style={{
 top: `${(r1 / 8) * 100}%`,
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
 />

 {/* Показываем 4 угла в самой матрице */}
 <div className="absolute pointer-e
 shadow-[0_0_10px_#f43f5e]" style={{ top: `
lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
 <div className="absolute pointer-e
 shadow-[0_0_10px_#3b82f6]" style={{ top: `
 width: `calc(`${(Ly / 8) * 100}% - 16px)`,
 <div className="absolute pointer-e
 ld-400 shadow-[0_0_10px_#10b981]" style={{
 8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
 <div className="absolute pointer-e

```



роса. (Цветные прямоугольники слева).  
</p>

```

 <div className="bg-[#0a0fle] rounded-xl
text-slate-300 shadow-inner overflow-x-auto
 <div className="absolute right-3 top-3
 // 1.
 int<
ext-amber-300">1); <span className="
 int<
ext-amber-300">1); <span className="
 int<
"text-slate-500">// Выс: {Lx}

 int<
"text-slate-500">// Шир: {Ly}
<b

 // 2.
 int<

В запроса)

r1][<span className="text-white bg-r

во, чтобы правый край уперся в c2)<b

r1][<span className="text-white bg-b

рх, чтобы нижний край уперся в r2)<b

nded">r2 - Lx + 1][<span className="

и влево от c2)

">r2 - Lx + 1][<span className="text
{'}}'}});

```

```
)
 }}}}
 </div>
 </div>
</div>

 <div className="mt-auto bg-slate-950 font-sans text-white
)]">
 <div className="text-center font-bold text-white
wrap bg-[#0a0f1e] py-3 px-2 rounded-lg border-2 border-rose-500">

 n>

 span>

 + 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly]
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
)
 }
 }
```

---

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

---

---

ФАЙЛ 71/81: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

---

```
function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-');
 body.classList.add('theme-' + theme);

 // Just some visual feedback on buttons
 document.querySelectorAll('[id^="theme-"]')
 .forEach(btn => btn.classList.remove('outline', 'outline-'));

 if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('bg-dark', 'text-white', 'transition-colors');
 if (theme === 'light') document.getElementById('btn-theme-light').classList.add('bg-light', 'text-black', 'transition-colors');
 if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('bg-terminal', 'text-white', 'transition-colors');

 document.getElementById('btn-theme-dark').classList.add('bg-dark', 'text-white', 'transition-colors');
 document.getElementById('btn-theme-light').classList.add('bg-light', 'text-black', 'transition-colors');
 document.getElementById('btn-theme-notebook').classList.add('bg-notebook', 'text-white', 'transition-colors');
}

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer');
 const drawerOverlay = document.getElementById('drawer-overlay');

 function toggleDrawer() {
 mobileMenuBtn.classList.toggle('active');
 mobileDrawer.classList.toggle('active');
 closeDrawerBtn.classList.toggle('active');
 drawerOverlay.classList.toggle('active');
 }
});
```

```
 const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
 };

 const observer = new IntersectionObserver((
 entries.forEach(entry => {
 if (entry.isIntersecting && !document
 // Highlight active TOC link lo
 const id = entry.target.id;
 document.querySelectorAll('aside
 if (link.getAttribute('href
 link.classList.add('bg-
 } else {
 link.classList.remove('
 }
 }));
 }
 }));
 }, observerOptions);

 document.querySelectorAll('section[id^="que
 observer.observe(section);
 });
 });
</script>
</body>
</html>
```

---

ФАЙЛ 72/81: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР

---

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
```

```
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
 }
 .formula-scroll::-webkit-scrollbar {
 display: none;
 }

 mjx-container {
 max-width: 100% !important;
 }
 mjx-container[display="true"] {
 overflow-x: auto;
 overflow-y: hidden;
 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
 }
 mjx-container[display="true"]::-webkit-scrollbar
 display: none;
 }

 @media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font
 .uno-card::before, .uno-card::after { font-
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px;

 .uno-card.mini { width: 28px !important; he
 us: 4px; }
 .uno-card.mini::before, .uno-card.mini::aft

 .uno-equation .text-2xl { font-size: 1.2rem
```

```

 important; }
body.theme-light .text-slate-200, body.theme-light .text-slate-400 { color: #475568; }
body.theme-light .border-slate-600, body.theme-light .border-slate-500 { border-color: #475568; }
body.theme-light #top-controls { background-color: #f8f8f8; }

body.theme-notebook {
 background-color: #fcf8eb !important; /* light beige */
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient(45deg, transparent, transparent 2px, #3b2818 2px, #3b2818 4px);
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook .border-slate-700 {
 background-color: rgba(255, 255, 255, 0.7);
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-400 { color: #475568; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-500 { border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Monospace"; }
body.theme-notebook #top-controls { background-color: #f8f8f8; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1); }
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
}

```

```
</div>
```

```
<!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
```

```
<svg width="0" height="0" class="absolute">
```

```
 <defs>
```

```
 <marker id="arrow-yellow" markerWidth="6" m
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d
```

```
 </marker>
```

```
 <marker id="arrow-orange" markerWidth="6" m
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#f9731d
```

```
 </marker>
```

```
 <marker id="arrow-pink" markerWidth="6" mar
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#f472b1
```

```
 </marker>
```

```
 <marker id="arrow-cyan" markerWidth="6" mar
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf
```

```
 </marker>
```

```
 <marker id="arrow-indigo" markerWidth="6" m
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8
```

```
 </marker>
```

```
 <marker id="arrow-lime" markerWidth="6" mar
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d
```

```
 </marker>
```

```
 <marker id="arrow-green" markerWidth="6" ma
```

```
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade88
```

```
 </marker>
```

```
<!-- ЛЕГО БЛОКИ -->
```

```
<g id="lego-yellow">
```

```
 <rect x="0" y="0" width="40" height="20
```

```
 <rect x="6" y="-5" width="8" height="6"
```

```
 <rect x="26" y="-5" width="8" height="6"
```

```
 <line x1="2" y1="2" x2="38" y2="2" stro
```

```
</g>
```

```
<g id="lego-red">
```

```
 <rect x="0" y="0" width="40" height="20
```

```
 <rect x="6" y="-5" width="8" height="6"
```

```
 <rect x="26" y="-5" width="8" height="6"
```

```

 </svg>
 </button>

 <!-- Затемнение фона на мобилках при открытом меню -->
 <div id="drawer-overlay" class="lg:hidden fixed inset-0
 background-color-slate-500/50 transition-colors duration-300"
 >

 <div class="max-w-7xl mx-auto mt-10 px-4 flex flex-col items-center"
 >

 <!-- Левое меню (Оглавление) -->
 <aside id="mobile-drawer" class="fixed inset-y-0 left-0
 w-full bg-slate-x-full transition-transform duration-300 ease-in-out
 w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto" >
 <div class="h-full flex flex-col p-6 overflow-y-auto" >
 <div class="flex justify-between items-center" >
 <h3 class="font-black text-transparent bg-clip-text
 text-slate-900" >
 Содержание
 </h3>
 <button id="close-drawer-btn" class="text-slate-500
 lg:hidden" >
 <svg xmlns="http://www.w3.org/2000/svg"
 class="h-6 w-6" >
 <path stroke-linecap="round" stroke-width="2"
 d="M6 18 18 6M6 6 18 18" >
 </svg>
 </button>
 </div>
 <nav class="space-y-3 text-sm font-medium" >
 Главная
 <details class="group" open>
 <summary class="list-none cursor-pointer" >
 <a href="#question-1" class="text-blue-500
 pl-3 font-bold text-blue-300" >
 1. Алгебраические структуры

 </summary>
 <div class="pl-6 space-y-2 text-sm" >
 1.1. Группы
 1.2. Кольца
 1.3. Модули
 </div>
 </details>
 </nav>
 </div>
 </aside>
 </div>

```



```


 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q4-def" class="block
 <a href="#q4-moivre" class="blo
 <a href="#q4-root" class="block
 <a href="#q4-count" class="bloc
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-5" class:
er-fuchsia-500 pl-3 font-bold text-fuchsia-
 5. Группа корней из единицы

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q5-def" class="block
 <a href="#q5-group" class="bloc
 <a href="#q5-prim" class="block
 <a href="#q5-crit" class="block
 <a href="#q5-cyclic" class="blo
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-6" class:
er-lime-500 pl-3 font-bold text-lime-300">
 6. Делимость, НОД и Евклид

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q6-1" class="block ho
 <a href="#q6-2" class="block ho
 <a href="#q6-3" class="block ho
```

```
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-emerald-green-500 pl-3 font-bold text-green-400">
 9. Кольцо многочленов

</summary>
<div class="pl-6 space-y-2 text-emerald-green-500">

</div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-pink-rose-500 pl-3 font-bold text-rose-400">
 10. Делимость и Схема Г

</summary>
<div class="pl-6 space-y-2 text-pink-rose-500">

</div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-slate-cyan-500 pl-3 font-bold text-cyan-400">
```

```


 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold">

 14. Кратность корня. Коэффициент

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 deg)
 K
 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold">

 15. Производная. Критерий

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
</details>

<details class="group">
```

```
 <a href="#proof-algorithms" class="link
-red-500 pl-3 mt-4 text-red-500 font-bold m
 ▲ Ликбез: Алгоритмы выживания

```

```
</nav>
```

```
<script>
```

```
 document.querySelectorAll('details.'
 detail.addEventListener('toggle
 if (detail.open) {
 document.querySelectorA
 if (other !== detail
 other.removeAtt
 }
 });
 }
 });
 });
```

```
 // Scroll spy logic
 document.addEventListener('DOMConten
 const sections = document.query
 ="q5-"], div[id^="q6-"], div[id^="q7-"], di
 v[id^="q13-"], div[id^="q14-"], div[id^="q1
 let isScrolling = false;
```

```
 document.querySelectorAll('.gro
 link.addEventListener('clie
 isScrolling = true;
 setTimeout(() => isScro
 });
 });
```

```
 const observer = new Intersection
 if (isScrolling) return;

 let visibleId = null;
```

```
function fallbackModal(contentRaw,
 const modal = document.createElement('div');
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';
 modal.style.height = '100vh';
 modal.style.backgroundColor = 'black';
 modal.style.zIndex = '99999';
 modal.style.display = 'flex';
 modal.style.flexDirection = 'column';
 modal.style.alignItems = 'center';
 modal.style.justifyContent = 'center';

 const box = document.createElement('div');
 box.style.width = '80%';
 box.style.height = '80%';
 box.style.backgroundColor = '#1a1a1a';
 box.style.padding = '20px';
 box.style.borderRadius = '10px';
 box.style.display = 'flex';
 box.style.flexDirection = 'column';
 box.style.color = 'white';

 const header = document.createElement('div');
 header.innerText = 'Экспорт в PDF';
 header.style.marginBottom = '10px';

 const subHeader = document.createElement('div');
 subHeader.innerText = 'Из-за проблем с совместимостью
некоторые веб-страницы могут отображаться некорректно.
Пожалуйста, попробуйте обновить браузер или использовать
альтернативный способ экспорта (например, через меню)';
 subHeader.style.marginBottom = '10px';
 subHeader.style.color = '#f87171';
 subHeader.style.fontSize = '0.9em';

 const textarea = document.createElement('div');
 textarea.value = contentRaw;
 textarea.style.flex = '1';
```

```

 closeBtn.onclick = () => modal.

 box.appendChild(header);
 box.appendChild(subHeader);
 if (title !== 'PDF') {
 box.appendChild(textarea);
 btnContainer.appendChild(co
 }
 btnContainer.appendChild(closeB
 box.appendChild(btnContainer);
 modal.appendChild(box);
 document.body.appendChild(modal
 }

function downloadMD() {
 let clone = document.querySelec

 // Smart markdown parsing based

 // Smart markdown parsing based
 clone.querySelectorAll('h2').fo
 clone.querySelectorAll('h3').fo
 clone.querySelectorAll('.analog
 clone.querySelectorAll('li').fo
 clone.querySelectorAll('svg').f
 let caption = '';
 if (el.nextElementSibling &
 caption = el.nextElement
 }
 el.textContent = '\n\n[Илл
 });
 clone.querySelectorAll('.img-ca

 let text = clone.innerText;
 text = text.replace(/\n{3,}/g,

 if (window !== window.top) {
 fallbackModal(text, 'Markdo

```

```
/* Universal text color */
body.bg-white .text-slate-900 {
 color: #1e293b;
}

/* Specific backgrounds */
body.bg-white aside > div {
 background-color: #f8fafc;
}

body.bg-white [class*=bg] {
 background-color: #f8fafc;
}

body.bg-white [class*=card] {
 background-color: #fffbeb;
}

body.bg-white .analogy-block {
 background-color: #fffbeb;
}

body.bg-white .img-placeholder {
 background-color: #fffbeb;
}

body.bg-white .info-block {
 background-color: #fffbeb;
}

body.bg-white .analogy-block {
 background-color: #fffbeb;
}

or: #94a3b8 !important; }
body.bg-white .img-caption {
 background-color: #fffbeb;
}

/* Tweak card blocks */
body.bg-white .card-red {
 background-color: #fffbeb;
}

body.bg-white aside {
 background-color: #fffbeb;
}

#334155 !important; }
body.bg-white .text-white {
 color: white;
}

/* Ensure active links are visible */
body.bg-white aside a.text-white {
 color: white;
}
body.bg-white aside a.text-white {
 color: white;
}
body.bg-white aside a.text-white {
 color: white;
}
body.bg-white aside a.text-white {
 color: white;
}
body.bg-white aside a.text-white {
 color: white;
}
body.bg-white aside a.text-white {
 color: white;
}
body.bg-white aside a:hover {
 color: white;
}

body.bg-white table tr {
 background-color: #fffbeb;
}
```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И
<div class="mt-8 flex flex-col gap-4 bo
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">
 <div class="text-xs text-slate-
 <button onclick="downloadWord()
ont-bold flex items-center justify-center g
 Word
 </button>
 <a href="algebra.pdf" target="_l
-sm font-bold flex items-center justify-cen
 PDF

 <button onclick="downloadMD()"
-bold flex items-center justify-center gap-
 Markdown
 </button>
 <button onclick="downloadHTML()
t-sm font-bold flex items-center justify-ce
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-
 <div class="flex items-center g
 <button id="theme-dark" onc
justify-center text-amber-300 hover:text-wh
line-amber-500" title="Темная тема"> </butt
 <button id="theme-light" on
r justify-center text-amber-50 hover:text-wh
">* </button>
 <button id="theme-terminal"
-center justify-center text-green-400 hover
терминала"> </button>

```



```

 18.
 19.
 20.
 21.
 22.

</div>
</header>

<!-- ===== WTF CONTAINER =====>
<div id="wtf-container" class="hidden">
 <h2 class="text-3xl font-black text-cen

 <div class="overflow-x-auto w-full pb-4">
 <div class="grid grid-cols-3 gap-4">
 <!-- Column headers -->
 <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
 <div class="bg-orange-900/40 bo
ольцо полиномов</div>
 <div class="bg-cyan-900/40 bord
сные числа</div>

 <!-- Row 1: База -->
 <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slat

 <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
 <div class="text-xs text-orange
 <div class="font-bold text-slat

 <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slat

```

```


 <a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 5: Корни / Уравнения -->
 <div class="hidden md:block"></div>
 <a href="#question-13" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>
 <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>
 <a href="#question-15" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t

```



```
<!-- Индукция -->
<div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-yellow">
 <p class="text-sm text-slate">
 <ul class="list-disc pl-5">

 валось.

 Просто перепиши формулу с буквой k .

 усок k , замени его на жареного карася из

 </div>
```

```
<!-- Двойная делимость -->
<div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-orange">
 <p class="text-sm text-slate">
 <ul class="list-disc pl-5">

 вверх по уравнениям).

 рху вниз, показывая, что $\Delta \vdots r_i$

 х. Твой d больше.

 </div>
```

```
<h2 class="text-2xl font-bold">
 ритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md">
 <div class="bg-slate-800 p-6">
 <h4 class="text-red-400">
 <p class="text-sm text-slate">
 <p class="text-xs font-
```

```

 <h4 class="text-cyan">
 <p class="text-sm">
 <code class="text-x">
 </div>

<li class="bg-slate-800/50">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 x двух, потом последних.

 <code class="text-x">
 </div>

 <li class="bg-slate-800/50">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 есть сложение – есть ноль. Если есть умноже
 <code class="text-x">
 </div>

 <li class="bg-slate-800/50">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 авь их драться. Умножение всегда врывается
 <code class="text-x">
 </div>

 </div>
</section>
```

```
* Живые React-визуализации в статику не переносятся —
* подставляется заметка со ссылкой на онлайн-версию, а
* которые в приложении всплывают по наведению, собираю
* в конце документа.
*/
```

```
/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
 parts.push(Array.from(rules).map((r) => r.cssText));
 } catch {
 const href = (sheet as CSSStyleSheet).href;
 if (!href) continue;
 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен — пропускаем */
 }
 }
 }
 return parts.join("\n");
}
```

```
const EXTRA_CSS = `
/* — правки для автономного файла — */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; paddi
.export-head { border-bottom: 1px solid #1e293b; paddin
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
```

```

 const link = document.createElement("a");
 link.className = "term-hint";
 link.href = `#term-${id}`;
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
 link.textContent = el.textContent ?? "";
 el.replaceWith(link);
 });

 // <details> в файле лучше раскрыть: иначе код и разб
 host.querySelectorAll("details").forEach((d) => d.set

 return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escape
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal
${e.complexity ? `<p class="cx">Сложность: ${escapeHt
</div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
<h2>Словарик терминов из этой темы</h2>
<p style="font-size:.8rem;color:#94a3b8;margin:0 0 1r
 ны сюда.</p>
 ${items}
</section>`;
}

function wrap(opts: { title: string; subtitle: string;

```

```
const url = URL.createObjectURL(blob);
const a = document.createElement("a");
a.href = url;
a.download = filename;
document.body.appendChild(a);
a.click();
a.remove();
setTimeout(() => URL.revokeObjectURL(url), 2000);
}

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
 const num = title.match(/^\\s*(\\d+)/)?.[1];
 const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
 return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}

/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
 const css = await collectCss();
 const { body, termIds } = prepareBody(chapter.content);
 const html = wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
 triggerDownload(html, safeName(chapter.title, chapter.id));
}

/** Выгрузить всё пособие одним файлом со сквозным оглавлением
export async function downloadBookHtml(chapters: Chapter[]) {
 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];
 for (const chapter of chapters) {
 const { body, termIds } = prepareBody(chapter.content);
 allTerms.push(...termIds);
 bodies.push(body);
 }
 const gloss = glossarySection(allTerms);
 const html = wrap({
 title: "Универсальное пособие",
 subtitle: "Содержание",
 css,
 body: bodies.join("\n"),
 gloss,
 origin: window.location.origin,
 });
 triggerDownload(html, "Универсальное пособие.html");
}
```



```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\s{2

const rows = chapters.map((c) => {
```

```

 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(` * ${r.title} [$
 console.log();
 });
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
 dump("НЕТ 2D (аналогия есть)", gapViz);
 console.log(`Пропущенные номера билетов 1-24: ${missi
 console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

ФАЙЛ 76/81: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```

/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossa
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".m
 external: ["react", "react-dom", "motion", "lucide-re
});

const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY_LABELS, GLOSSARY } = await import(path

const esc = (s) => s.replace(/[\.*\+?\^\$\{\}\(\)|[\]\[\]]/g, "\\

```

```
 console.log(
 String(ids.size).padStart(3), "|", String(highlight
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0,
));
}
console.log("\nГлав без единой подсказки:", bad.length,
console.log("Терминов в глоссарии:", GLOSSARY.length, "
console.log("Не встретились:", GLOSSARY.filter((g) => !
```

---

ФАЙЛ 77/81: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ

---

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`
 }

 const pages = fs
 .readdirSync(PAGES_DIR)
 .filter((f) => f.endsWith(".png"))
 .sort();
```



```

 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 walk(ROOT);
}

return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
 .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
 .filter((rel) => CODE_EXT.has(path.extname(rel)))
 .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 }));
}

/** Достаём заголовки глав из данных пособия (без запуска) */
function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("src/"))) {
 const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
 }
}

```

```

push("
push(HR);
let current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push();
 push(`  # ${cat}`);
 }
 push(`  ${String(i + 1).padStart(3, " ")}: ${rel}`);
});
push();
push();

```

```

current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 push(HR);
 push();
 }
 const content = fs.readFileSync(path.join(ROOT, rel));
 const lines = content.split("\n");
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} | `);
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
});

```

```

push(HR);

```

Универсальное пособие • полный исходный код

```

export const PAGES_DIR = path.join(ROOT, "tmp", "export

export const TXT_PATH = path.join(OUT_DIR, "full_code_a
export const PDF_PATH = path.join(OUT_DIR, "full_code_a

/** Параметры вёрстки страницы (подобраны под A4 @150dp
export const PAGE = {
 /** DPI раstra. Можно переопределить: EXPORT_DENSITY=
 density: Number(process.env.EXPORT_DENSITY ?? 150),
 /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT
 monochrome: process.env.EXPORT_GRAYSCALE !== "1",
 size: "A4",
 font: "DejaVu-Sans-Mono",
 fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan
 pointsize: 8,
 /** Максимум символов в строке (далее – жёсткий пере
 cols: 138,
 /** Строк содержимого на странице (+2 служебные: шапка
 rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 for (const cmd of ["magick", "convert"]) {
 try {
```

```

const expanded = line.replace(/\t/g, " ").replace(
if (expanded.length <= PAGE.cols) return [expanded];
const parts = [];
const indent = " ".repeat(Math.min((expanded.match(/^
let rest = expanded;
let first = true;
while (rest.length > 0) {
 const width = first ? PAGE.cols : PAGE.cols - indent;
 parts.push((first ? "" : indent) + rest.slice(0, width));
 rest = rest.slice(width);
 first = false;
}
return parts;
}

function paginate(txt) {
 const source = txt.replace(/\r\n/g, "\n").split("\n");
 const pages = [];
 for (let i = 0; i < source.length; i += PAGE.rows) {
 pages.push(source.slice(i, i + PAGE.rows));
 }
 return pages;
}

async function main() {
 if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Her ${path.relative(ROOT, TXT_PATH)}`);
 }

 const im = imageMagickCmd();
 const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

 fs.rmSync(PAGES_DIR, { recursive: true, force: true });
 ensureDir(PAGES_DIR);

 const total = pages.length;
 const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");

```



```
 }
 }

 await Promise.all(Array.from({ length: concurrency },
 process.stdout.write("\n"));

 const missing = jobs.filter((j) => !fs.existsSync(j.path));
 if (missing.length) throw new Error(`Не отрендерены следующие файлы: ${missing.map(j => j.path).join(", ")}`);

 const bytes = jobs.reduce((s, j) => s + fs.statSync(j.path).size, 0);
 console.log("[2/3] txt -> png");
 console.log(`      страниц: ${total}`);
 console.log(`      выход:   ${path.relative(ROOT, PAGE_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка рендера страниц:", err.message);
 process.exit(1);
});
```

---

## РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

---

---

ФАЙЛ 81/81: [.github/workflows/rebuild-pdf.yml](#)

КАТЕГОРИЯ: CI / автоматизация | СТРOK: 128 | РАЗМЕР: 4.0 КБ

---

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
└───────────→ page-0001.png ... page-NNNN.png (images)
#
```



```
.....
 echo "Экспорт не изменился – коммит не нужен"
 exit 0
fi
git commit -m "chore(export): пересборка full"
git push origin "HEAD:${GITHUB_REF_NAME}"
```